



République tunisienne

Ministère de l'Enseignement supérieur
et de la Recherche scientifique

École Privée Supérieure d'Ingénierie et de Technologie

TEK-UP



RAPPORT DE PROJET DE FIN D'ÉTUDES

en accomplissement partiel des exigences pour l'obtention du diplôme de

Diplôme National d'Ingénieur en Sciences Appliquées et Technologie

Specialization: Génie logiciel et systèmes d'information

Préparé par

Mohamed Aziz Merai

Plateforme intelligente de recommandation et d'optimisation de L'infrastructure IT

Encadrant professionnel: **Monsieur Mehdi Kaouech**

Encadrant académique: **Madame Inès AGREBI**

CEO RedSys

Enseignante encadrante

— TEK-UP University

J'autorise l'étudiant à soumettre son rapport de stage en vue de la soutenance.

Professional Supervisor, **Mr. Mehdi Kaouech**

Signature and Stamp

J'autorise l'étudiant à soumettre son rapport de stage en vue de la soutenance..

Academic Supervisor, **Mme. Ines Agerbi**

Signature

Dédicace

À mes parents,

les deux seules personnes qui me connaissent depuis mon premier souffle

Vous avez tout donné à un seul enfant—

vos rêves, vos sacrifices, vos nuits blanches, vos prières silencieuses.

Vous m’avez offert le monde, sans rien demander en retour, si ce n’est mon bonheur.

À mes amis,

les frères et sœurs que je n’ai jamais eus.

Vous avez comblé le silence, partagé le poids,

et fait en sorte que j’aie toujours quelqu’un à appeler.

Vous êtes ma famille de cœur, mon rire, mon refuge.

À mes parents, encore — parce que vous méritez d’être cités deux fois,

chaque page de ce travail porte votre nom à l’encre invisible.

Chaque réussite est la vôtre.

Chaque pas en avant est possible parce que vous ne m’avez jamais laissé marcher seul. *À ceux*

qui sont partis trop tôt,

vous n’êtes pas là pour tenir ce diplôme avec moi,

mais je vous porte dans chaque battement de mon cœur et dans chaque victoire.

Ce travail est autant le vôtre que le mien.

Ceci est pour vous.

Remerciements

Avec une profonde gratitude...

Je tiens à exprimer ma plus profonde et sincère gratitude à toutes les personnes qui, par leurs conseils, leur soutien et leurs encouragements, ont contribué à la réussite de ce projet.

Tout d'abord, j'adresse mes plus vifs remerciements à mon encadrant professionnel,

Monsieur Mehdi Kaouech

dont l'accompagnement précieux, les remarques constructives et le soutien constant ont été une véritable source d'inspiration tout au long de ce stage. Son expertise et son engagement ont grandement enrichi ce travail.

Je suis également profondément reconnaissant envers mon encadrant académique,

Madame Inès AGREBI

pour son suivi attentif, ses encouragements constants et ses précieux conseils académiques. Sa patience et sa confiance en mes capacités ont été très motivantes.

J'adresse également mes sincères remerciements à toute l'équipe de

RedSys

pour leur accueil chaleureux, leur excellente collaboration et leur aide continue durant mon stage. L'environnement convivial et stimulant qu'ils ont offert a rendu cette expérience à la fois enrichissante et mémorable.

Table of Contents

List of Abbreviations	ix
Introduction générale	1
1 Contexte et Objectifs du Projet	3
1.1 Introduction	4
1.2 Présentation de l'organisme d'Accueil	4
1.2.1 Présentation de la Société RedSys	4
1.2.2 Mission et Vision de l'Entreprise	4
1.2.3 Organigramme	5
1.3 Présentation du projet	5
1.3.1 Etude de l'existant	5
1.3.2 Critique de l'existant	6
1.3.3 Solution proposée	6
1.4 Présentation de la méthodologie de gestion de projet	7
1.4.1 Les roles SCRUM	8
1.4.2 Les événements Scrum	8
1.4.3 Les artefacts de Scrum	9
1.5 Présentation du langage de modélisation	10
1.6 Conclusion	11
2 Analyse et spécification des besoins	12
2.1 Introduction	13
2.2 Spécification des besoins	13
2.2.1 Identification des acteurs	13
2.2.2 Spécification des besoins fonctionnels	13
2.2.3 Spécification des besoins non fonctionnels	15
2.3 Diagramme de cas d'utilisation global	15
2.4 Diagramme de Classes	16
2.5 Structure et découpage du projet	17
2.5.1 Identification de l'équipe SCRUM	17
2.5.2 Backlog de produit	18

2.5.3	Planification des Sprints	23
2.6	Architecture de l'Application	26
2.6.1	Architecture Physique	27
2.6.2	Architecture Logique	28
2.7	Choix Techniques et Environnement de Développement	29
2.7.1	Langages de Programmation	29
2.7.2	Couche API	31
2.7.3	Frameworks et Bibliothèques	32
2.7.4	Base de Données et ORM	34
2.7.5	DevOps et Déploiement	35
2.7.6	Outils de Développement	36
2.8	Conclusion	38
3	Release 1 : MVP	39
3.1	Introduction	40
3.2	Release 1 Backlog	40
3.3	Refined Use Case Diagram for Release 1	42
3.4	Class Diagram of Release 1	43
3.5	Sprint 1 : Authentication and User Management	43
3.5.1	Sprint 1 Objectives	43
3.5.2	Refined Use Case Diagram for Sprint 1	43
3.5.3	Analysis and Design for Sprint 1	44
3.5.4	Realization of Sprint 1	48
3.6	Sprint 2 : Mission Management and Mobile Application	50
3.6.1	Sprint 2 Objectives	50
3.6.2	Refined Use Case Diagram for Sprint 2	50
3.6.3	Analysis and Design for Sprint 2	50
3.6.4	Realization of Sprint 2	54
3.7	Sprint 3 : Data Collector Module	56
3.7.1	Sprint 3 Objectives	56
3.7.2	Refined Use Case Diagram for Sprint 3	56
3.7.3	Analysis and Design for Sprint 3	56
3.7.4	Realization of Sprint 3	58
3.8	Conclusion	59

4 Release 2 : Improvements	60
4.1 Introduction	61
4.2 Release 2 Backlog	61
4.3 Refined Use Case Diagram for Release 2	62
4.4 Class Diagram of Release 2	63
4.5 Sprint 4 : Web Supervision Dashboard and Alerts & Notifications	63
4.5.1 Sprint 4 Objectives	63
4.5.2 Refined Use Case Diagram for Sprint 4	63
4.5.3 Analysis and Design for Sprint 4	64
4.5.4 Realization of Sprint 4	70
4.6 Sprint 5 : AI and Multi-tenancy	72
4.6.1 Sprint 5 Objectives	72
4.6.2 Refined Use Case Diagram for Sprint 5	72
4.6.3 Analysis and Design for Sprint 5	72
4.6.4 Realization of Sprint 5	76
4.7 Conclusion	78
General Conclusion	79
Webography	81

List of Figures

1.1	Logo RedSys	4
1.2	Organigramme de RedSys	5
1.3	Processus SCRUM	10
1.4	Logo UML uml_logo	11
2.1	Diagramme de cas d'utilisation global	16
2.2	Diagramme de classes	17
2.3	Physical Architecture (N-Tier)	28
2.4	Architecture Logique (Three-Tier) — Next.js	28
2.5	Logo HTML5	29
2.6	Logo CSS3	30
2.7	Logo JavaScript	30
2.8	Logo TypeScript	30
2.9	Logo Python	31
2.10	Illustration API REST	31
2.11	Logo FastAPI	32
2.12	Logo Next.js	32
2.13	Logo React	33
2.14	Logo Tailwind CSS	33
2.15	Logo shadcn/ui	33
2.16	Logo PostgreSQL	34
2.17	Logo Prisma ORM	34
2.18	Logo Docker	35
2.19	Logo Docker Compose	35
2.20	Logo GitHub Actions	36
2.21	Logo Vercel	36
2.22	Logo Visual Studio Code	36
2.23	Logo Postman	37
2.24	Logo pgAdmin	37
2.25	Logo Git	38
2.26	Logo GitHub	38

3.1	Refined Use Case Diagram – Release 1	42
3.2	Class Diagram – Release 1	43
3.3	Refined Use Case Diagram – Sprint 1	44
3.4	Sequence Diagram – Login	46
3.5	Sequence Diagram – Add User	48
3.6	Sprint 1 – Login Interface	49
3.7	Sprint 1 – User Management Interface	49
3.8	Refined Use Case Diagram – Sprint 2	50
3.9	Sequence Diagram – Add Mission	52
3.10	Sequence Diagram – Update Mission Status	54
3.11	Sprint 2 – Mission Management Interface	55
3.12	Sprint 2 – Technician Mobile Application	55
3.13	Refined Use Case Diagram – Sprint 3	56
3.14	Sequence Diagram – Data Collection	58
3.15	Sprint 3 – Data Collector Monitoring Interface	59
4.1	Refined Use Case Diagram – Release 2	62
4.2	Class Diagram – Release 2	63
4.3	Refined Use Case Diagram – Sprint 4	64
4.4	Sequence Diagram – View Station Status	66
4.5	Sequence Diagram – Receive Alert	68
4.6	Sequence Diagram – Push Notification	70
4.7	Sprint 4 – Web Supervision Dashboard	71
4.8	Sprint 4 – Alert Management Interface	71
4.9	Refined Use Case Diagram – Sprint 5	72
4.10	Sequence Diagram – AI-Based Anomaly Detection	74
4.11	Sequence Diagram – Provision New Tenant	76
4.12	Sprint 5 – AI Anomaly Detection Dashboard	77
4.13	Sprint 5 – Multi-Tenant Management Portal	77

List of Tables

2.1	Backlog de produit - Plateforme Intelligente de Recommandation de l'Infrastructure IT	18
2.1	Product Backlog	22
2.2	Planning des Sprints	26
3.1	Release 1 Backlog (MVP)	42
3.2	Textual Description – Use Case «Authenticate»	45
3.3	Textual Description – Use Case «User Management»	47
3.4	Textual Description – Use Case «Mission Management»	51
3.5	Textual Description – Use Case «Update Mission Status»	53
3.6	Textual Description – Use Case «Data Collection»	57
4.1	Release 2 Backlog (Improvements)	62
4.2	Textual Description – Use Case «Web Supervision Dashboard»	65
4.3	Textual Description – Use Case «Alert Management»	67
4.4	Textual Description – Use Case «Push Notification»	69
4.5	Textual Description – Use Case «AI-Based Anomaly Detection»	73
4.6	Textual Description – Use Case «Multi-Tenant Management»	75

Liste des abréviations

Abbreviation	Description
API	Application Programming Interface
ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
CI/CD	Continuous Integration / Continuous Deployment
DB	Database
DOM	Document Object Model
ELK	Elasticsearch, Logstash, Kibana
EV	Electric Vehicle
FCM	Firebase Cloud Messaging
FK	Foreign Key
GQL	GraphQL
GORM	Go Object Relational Mapper
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
IT	Information Technology
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MoSCoW	Must Have, Should Have, Could Have, Won't Have
MVC	Model View Controller
MVP	Minimum Viable Product
OCPP	Open Charge Point Protocol
OOP	Object Oriented Programming
ORM	Object Relational Mapper
OS	Operating System

Abbreviation	Description
PK	Primary Key
REST	Representational State Transfer
SaaS	Software as a Service
SDL	Schema Definition Language
SQL	Structured Query Language
SSL	Secure Sockets Layer
TLS	Transport Layer Security
TS	TypeScript
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience
W3C	World Wide Web Consortium
WS	WebSocket

Introduction générale

Le marché des équipements informatiques reconditionnés connaît une croissance significative, portée par les impératifs de réduction des coûts et les enjeux environnementaux croissants. Face à cette dynamique, les entreprises peinent pourtant à sélectionner les infrastructures serveur les mieux adaptées à leurs besoins, faute d'outils d'analyse et de simulation dédiés. La complexité technique inhérente au choix d'un serveur reconditionné – tenant compte des paramètres CPU, RAM, stockage et charge applicative – nécessite une expertise avancée que la plupart des organisations ne possèdent pas en interne.

Dans ce contexte, **RedSys**, entreprise spécialisée dans les solutions IT et la commercialisation d'équipements informatiques reconditionnés, a identifié la nécessité de moderniser son offre à travers une plateforme web intelligente. L'approche actuelle, reposant sur une présentation statique des produits sans outil d'aide à la décision, ne permet pas aux clients entreprises d'évaluer objectivement la pertinence d'un serveur par rapport à leurs besoins réels. Cette lacune impacte directement la satisfaction client, la performance commerciale de RedSys et la valorisation de son catalogue de produits reconditionnés.

L'objectif de ce projet est de concevoir et développer une plateforme web intelligente composée de plusieurs modules complémentaires : un moteur de recommandation automatique basé sur l'intelligence artificielle pour orienter les clients vers les équipements les plus adaptés à leur profil, un simulateur de performance permettant de tester des scénarios de charge, un tableau de bord analytique décisionnel à destination de l'équipe RedSys, un espace de gestion du stock et des commandes, ainsi qu'un système de ticketing pour le support client. La plateforme intègre également une refonte complète du site vitrine de RedSys, offrant une expérience utilisateur moderne et cohérente.

La solution repose sur une architecture technique moderne et évolutive, combinant **Next.js** pour le frontend et le backend, **PostgreSQL** comme système de gestion de base de données relationnelle interfacé via **Prisma**, et une **API REST sécurisée** avec authentification **JWT**. Le moteur de recommandation s'appuie sur un **service IA externe** intégré à la plateforme, garantissant des recommandations précises et personnalisées.

Le processus de développement suit la méthodologie agile **Scrum**, organisée en sprints itératifs, assurant une livraison progressive de valeur et un alignement continu avec les besoins évolutifs de l'entreprise.

Ce rapport est structuré comme suit :

-
- **Chapitre 1** présente le contexte du projet, l’entreprise d’accueil RedSys, les limites du système existant et la solution proposée. Il introduit également la méthodologie Scrum et le langage de modélisation UML adoptés tout au long du projet.
 - **Chapitre 2** couvre l’analyse et la spécification des besoins. Il identifie les acteurs du système, définit les exigences fonctionnelles et non fonctionnelles, établit le product backlog, détaille le planning des sprints, présente l’architecture du système aux niveaux physique et logique, et justifie les choix techniques retenus.
 - **Chapitre 3** détaille la conception et l’implémentation de la Release 1, couvrant les premiers sprints dédiés au site vitrine, au module de recommandation intelligente et au simulateur de performance.
 - **Chapitre 4** se concentre sur la Release 2, présentant les fonctionnalités des sprints suivants : le tableau de bord analytique, la gestion du stock et des commandes, ainsi que le système de ticketing et support client.

Ce travail représente une étape concrète vers la transformation de RedSys en une plateforme intelligente d’aide à la décision, positionnant l’entreprise comme un acteur innovant dans la valorisation des infrastructures IT reconditionnées et posant les bases d’une solution scalable, fiable et orientée vers l’avenir.

CONTEXTE ET OBJECTIFS DU PROJET

Plan

1	Introduction	4
2	Présentation de l'organisme d'Accueil	4
3	Présentation du projet	5
4	Présentation de la méthodologie de gestion de projet	7
5	Présentation du langage de modélisation	10
6	Conclusion	11

1.1 Introduction

Ce chapitre présente le contexte du projet, l'entreprise d'accueil, la problématique identifiée, ainsi que la méthodologie adoptée pour le développement de la plateforme intelligente de recommandation et d'optimisation de l'infrastructure IT.

1.2 Présentation de l'organisme d'Accueil

1.2.1 Présentation de la Société RedSys

RedSys est une entreprise spécialisée dans les solutions IT, la gestion d'infrastructure et la commercialisation d'équipements informatiques reconditionnés. Reconnue pour son expertise dans le domaine des serveurs et composants informatiques, l'entreprise accompagne les organisations dans l'optimisation de leurs infrastructures en proposant des équipements de qualité à des coûts maîtrisés, contribuant ainsi à une démarche à la fois économique et responsable sur le plan environnemental.



Figure 1.1: Logo RedSys

1.2.2 Mission et Vision de l'Entreprise

Mission: La mission de RedSys est d'être un partenaire de confiance pour les entreprises souhaitant optimiser leurs infrastructures informatiques à travers des équipements reconditionnés de haute qualité. En tant que spécialiste des solutions IT, l'entreprise s'engage à proposer des équipements fiables, des conseils techniques adaptés aux besoins de chaque client, et des services à forte valeur ajoutée, tout en respectant les valeurs d'excellence, d'innovation et de responsabilité environnementale.

Vision: RedSys aspire à devenir la référence incontournable dans le domaine des infrastructures IT reconditionnées, en se positionnant comme un acteur innovant capable d'accompagner les entreprises dans leurs décisions technologiques grâce à des outils intelligents d'aide à la décision. En intégrant l'intelligence artificielle dans son offre, RedSys entend créer une valeur durable pour ses clients, ses collaborateurs et l'ensemble de l'écosystème numérique.

1.2.3 Organigramme

La figure 1.2 illustre l’organigramme de RedSys

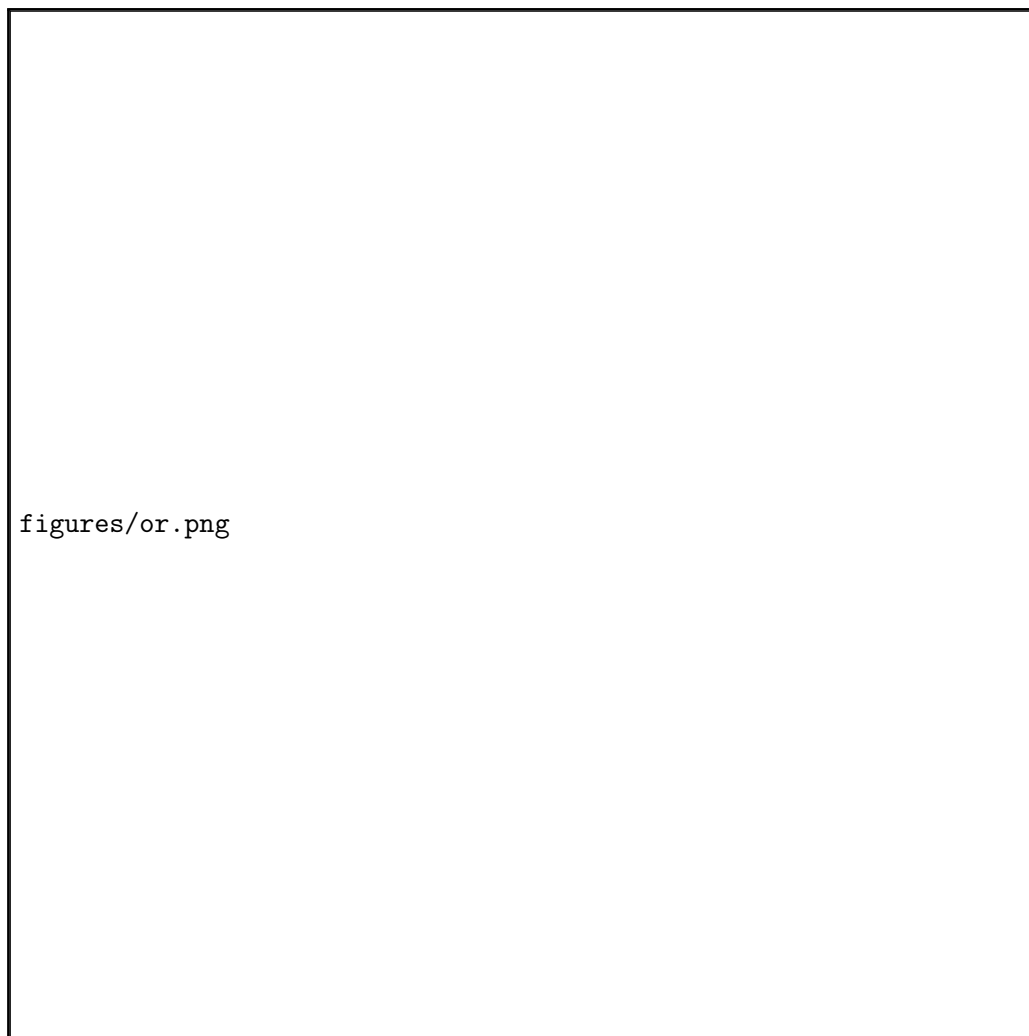


Figure 1.2: Organigramme de RedSys

1.3 Présentation du projet

Pour mener à bien ce projet, une étude préalable a permis de décrire l’existant puis d’en déterminer les limites afin de produire une solution qui répondra au mieux aux besoins de notre client.

1.3.1 Etude de l’existant

RedSys s’appuie actuellement sur un site web statique pour présenter son catalogue d’équipements informatiques reconditionnés et attirer des clients entreprises. Si cette approche a permis d’assurer une présence en ligne, elle ne propose aucun outil d’aide à la décision permettant aux clients d’évaluer la pertinence d’un équipement par rapport à leurs besoins réels. Plusieurs plateformes intégrées existent sur le marché et tentent de répondre à des besoins similaires, notamment :

- **Renwtech** – une plateforme de vente d’équipements informatiques reconditionnés proposant

- des fiches techniques détaillées et un catalogue structuré par catégories de produits ;
- **ServerMonkey** – une plateforme e-commerce spécialisée dans les serveurs reconditionnés offrant des fonctionnalités de filtrage avancé par marque, configuration et cas d’usage ;
- **Corelink IT** – une solution de gestion d’infrastructure IT intégrant des outils de recommandation et de suivi du cycle de vie des équipements pour les entreprises.

Cependant, aucune de ces solutions ne répond pleinement aux besoins spécifiques de RedSys, notamment la combinaison d’un moteur de recommandation intelligent basé sur l’IA, d’un simulateur de performance intégré et d’un tableau de bord analytique décisionnel pour la gestion interne.

1.3.2 Critique de l’existant

L’analyse des solutions existantes révèle plusieurs limites récurrentes dans le contexte actuel :

- **Absence d’aide à la décision** : Les plateformes actuelles présentent les équipements sous forme de catalogues statiques sans proposer de recommandations personnalisées basées sur les besoins réels du client (nombre d’utilisateurs, type d’application, charge de travail) ;
- **Absence de simulation de performance** : Aucune solution existante ne permet au client de simuler des scénarios de charge afin d’évaluer le comportement d’un serveur avant l’achat, ce qui engendre des risques de mauvais dimensionnement ;
- **Scalabilité limitée** : Les outils de gestion de catalogue utilisés ne sont pas conçus pour évoluer vers des fonctionnalités analytiques avancées, sans possibilité d’intégrer des indicateurs de performance ;
- **Absence de traçabilité et de pilotage interne** : Les solutions existantes ne proposent pas de tableau de bord permettant à RedSys de suivre les tendances de consultation, les configurations les plus demandées, ou les produits à faible rotation, ce qui limite la capacité de prise de décision stratégique ;
- **Faible support client et suivi des demandes** : L’absence d’un système de ticketing intégré contraint les clients à recourir à des canaux de communication génériques (email, téléphone) pour soumettre des demandes de support, sans possibilité de suivi structuré de l’état de leurs tickets.

1.3.3 Solution proposée

Pour pallier les limites identifiées, RedSys propose le développement d’une plateforme web intelligente et intégrée, composée de cinq modules principaux :

- **Un module de recommandation intelligente** – un moteur s’appuyant sur un service d’intelligence artificielle externe qui analyse le profil du client (nombre d’utilisateurs, type

d'application, besoins en stockage) et recommande automatiquement les équipements les plus adaptés, avec une estimation du score de performance ;

- **Un module de simulation de performance** – un outil interactif permettant au client de simuler des scénarios de charge de travail et d'évaluer le taux d'utilisation CPU et RAM, le niveau de risque et la stabilité estimée du serveur dans des conditions d'utilisation réelles ;
- **Un tableau de bord analytique décisionnel** – une interface dédiée à l'équipe RedSys permettant de visualiser en temps réel les configurations les plus demandées, les produits à forte rotation, les taux de consultation et les indicateurs stratégiques de performance commerciale ;
- **Un module de gestion du stock et des commandes** – un espace d'administration permettant la gestion CRUD des équipements, la mise à jour des informations techniques, et côté client, la gestion du panier, la modification des quantités et la validation des demandes d'achat ;
- **Un module de gestion des tickets de support** – un système structuré permettant aux clients de soumettre des demandes de support, de suivre l'état de leurs tickets, et à l'administrateur RedSys de consulter, répondre et clôturer les demandes selon leur degré d'urgence.

Cette architecture remplace l'approche statique actuelle par une infrastructure fiable, sécurisée et évolutive. Elle garantit une traçabilité de bout en bout, réduit considérablement les erreurs de dimensionnement, et positionne RedSys pour une croissance future à travers des évolutions planifiées telles que l'analyse prédictive du cycle de vie des équipements, la détection d'anomalies par IA et un modèle SaaS multi-clients.

1.4 Présentation de la méthodologie de gestion de projet

L'agilité représente une approche dynamique et adaptative de la gestion de projet et du développement logiciel, mettant l'accent sur la flexibilité, la collaboration et la capacité à s'ajuster aux évolutions des exigences, plutôt que de se conformer strictement à un plan préétabli. Cette méthodologie s'articule autour de cycles courts, appelés itérations, favorisant ainsi une amélioration continue et une livraison rapide de produits fonctionnels et de qualité. Parmi les diverses méthodes Agiles, **Scrum** se distingue comme l'une des plus répandues et appréciées. Scrum constitue un cadre de travail (framework) à la fois simple et puissant, conçu pour permettre aux équipes de collaborer efficacement tout en répondant aux besoins spécifiques des clients. Grâce à ses principes et pratiques bien définis, Scrum facilite la création de produits de haute qualité, tout en encourageant l'engagement et la responsabilité au sein

des équipes.

1.4.1 Les rôles SCRUM

On a trois rôles principaux en SCRUM :

Product Owner (PO) Le Product Owner joue un rôle central dans la méthodologie Scrum en agissant comme le principal représentant des parties prenantes et des clients. Il est responsable de la priorisation des éléments du backlog produit, garantissant que l'équipe de développement se concentre sur les fonctionnalités les plus précieuses et pertinentes. En collaborant avec les utilisateurs finaux et en recueillant leurs retours, le PO s'assure que le produit final répond aux attentes et aux besoins du marché.

SCRUM Master Le Scrum Master est le facilitateur du processus Scrum, veillant à ce que les principes et les pratiques de la méthodologie soient respectés. Il joue un rôle de coach pour l'équipe, en les aidant à s'améliorer et à adopter une mentalité Agile. Le Scrum Master favorise un environnement de travail collaboratif et productif, permettant ainsi à chaque membre de l'équipe de donner le meilleur de lui-même

Équipe de Développement Composée de professionnels aux compétences variées, l'équipe de développement est chargée de concevoir, développer et livrer les fonctionnalités du produit à la fin de chaque sprint. Cette équipe autonome collabore étroitement pour transformer les exigences du backlog en résultats tangibles. En s'engageant à respecter les délais et à fournir un produit de haute qualité, l'équipe de développement joue un rôle crucial dans la réussite du projet, tout en s'adaptant aux retours et aux changements de priorités au fil des itérations.

1.4.2 Les événements Scrum

Dans le cadre de la méthodologie Scrum, plusieurs événements clés structurent le processus de développement, permettant ainsi une gestion efficace des projets et une collaboration optimale au sein des équipes.

Sprint Le sprint est une itération qui dure généralement entre 1 et 4 semaines, durant laquelle l'équipe de développement s'engage à livrer un incrément fonctionnel du produit. Chaque sprint commence par une planification minutieuse et se termine par une évaluation, garantissant ainsi que le produit évolue de manière continue et répond aux attentes des utilisateurs.[1]

Daily Scrum (Mêlée quotidienne) Cette réunion quotidienne, d’une durée de 15 minutes, permet à l’équipe de se synchroniser et de discuter des progrès réalisés, des obstacles rencontrés et des tâches à accomplir. Le Daily Scrum favorise la transparence et la communication au sein de l’équipe, renforçant ainsi la cohésion et l’engagement de chacun.[2]

Sprint Planning (Planification du sprint) Lors de cette réunion de planification, l’équipe détermine les tâches à réaliser durant le sprint en fonction des priorités établies dans le backlog produit. Le Sprint Planning permet de clarifier les objectifs du sprint et d’assurer que chaque membre de l’équipe comprend son rôle et ses responsabilités, ce qui contribue à une exécution fluide et efficace.[3]

Sprint Review (Revue de sprint) À la fin de chaque sprint, une présentation est faite aux parties prenantes pour montrer le travail accompli. Cette réunion, appelée Sprint Review, permet de recueillir des retours précieux et d’ajuster les priorités pour les sprints futurs. Elle favorise également l’engagement des parties prenantes et leur implication dans le processus de développement. [4]

Sprint Retrospective (Rétrospective de sprint) Cet événement, qui se déroule après la Sprint Review, est dédié à l’amélioration continue des pratiques de l’équipe. Lors de la Sprint Retrospective, les membres de l’équipe réfléchissent ensemble sur ce qui a bien fonctionné, ce qui pourrait être amélioré et les actions à mettre en place pour optimiser leur collaboration et leur efficacité lors des prochains sprints. Ce processus d’auto-évaluation est essentiel pour favoriser une culture d’apprentissage et d’adaptation au sein de l’équipe. [5]

1.4.3 Les artefacts de Scrum

Dans Scrum, les artefacts servent à matérialiser soit les efforts fournis, soit la valeur produite. Ils permettent d’assurer une transparence maximale et offrent des occasions régulières d’inspection et d’adaptation. Conçus pour clarifier les informations essentielles, ils garantissent une compréhension commune au sein de toute l’équipe.

Backlog Produit Un backlog produit est une liste hiérarchisée de tâches destinées à l’équipe de développement. Il est créé à partir de la feuille de route produit et de ses exigences. Les éléments les plus importants figurent en tête du backlog produit. Ainsi, l’équipe sait ce qu’elle doit livrer en priorité. L’équipe de développement ne suit pas le backlog au rythme du Product Owner et celui-ci n’impose pas de tâches à l’équipe de développement. Cette dernière récupère les tâches à réaliser dans le backlog de produit en fonction de ses capacités, soit en continu (Kanban), soit par itération (Scrum).[6]

Backlog Sprint Le Backlog Sprint est un sous-ensemble du Backlog Produit, sélectionné et affiné pour être livré durant un Sprint donné. Il regroupe les éléments que l'équipe de développement s'engage à transformer en un incrément fonctionnel et potentiellement publiable d'ici la fin du Sprint. En plus des éléments choisis, le Backlog Sprint inclut un plan détaillé sur la manière dont le travail sera accompli, souvent représenté par des tâches techniques ou des sous-éléments définis par les développeurs eux-mêmes. Le Backlog Sprint constitue ainsi la prévision de l'équipe sur ce qu'elle pense pouvoir réaliser pendant l'itération en cours. Il reste cependant un artefact dynamique : l'équipe peut l'ajuster en cours de Sprint pour mieux atteindre l'objectif fixé. L'ensemble du travail intégré dans le Backlog Sprint doit, à l'issue du Sprint, satisfaire aux standards de qualité établis et être conforme à la définition de « Terminé » adoptée par l'équipe Scrum. [7]

Incrément L'incrément est constitué des éléments du Backlog Produit qui ont été jugés «Finis » durant le sprint, ainsi que de la valeur cumulative des incréments livrés lors des sprints précédents. À l'issue d'un Sprint, le nouvel incrément doit être « Fini », ce qui signifie qu'il doit être dans un état prêt à être publié et conforme à la définition de « Fini » (Definition of Done) établie par l'équipe de développement. Un incrément représente une portion de travail réalisée, qui soutient l'approche empirique et peut être vérifiée à la fin du Sprint. Il constitue une avancée vers une vision ou un objectif, et doit être dans un état prêt à être publié, indépendamment de la décision du Product Owner de le rendre public ou non.[8]

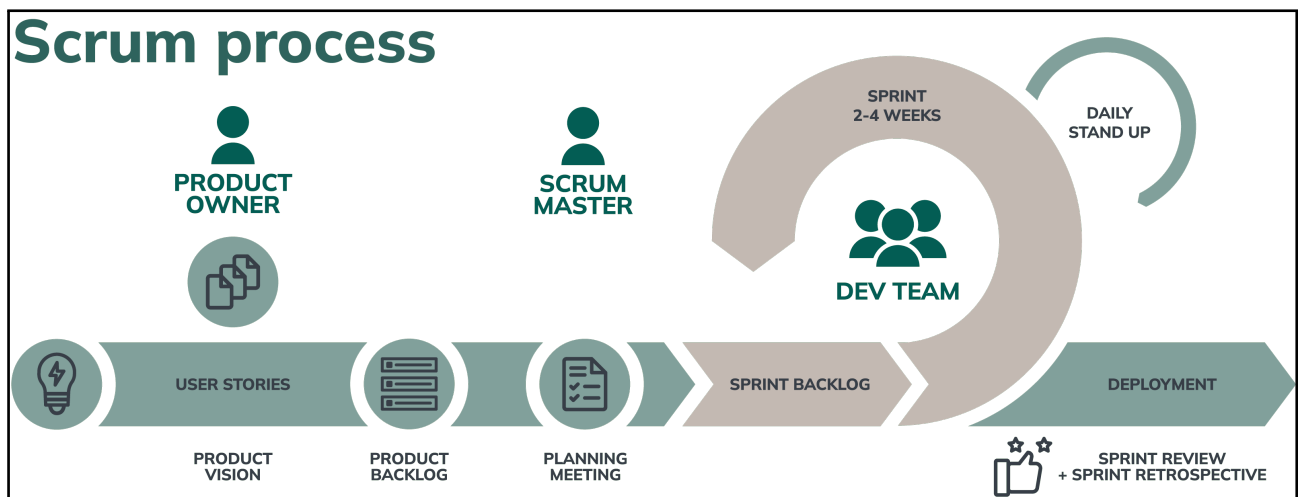


Figure 1.3: Processus SCRUM

1.5 Présentation du langage de modélisation

Ce projet repose sur les principes de la programmation orientée objet, ce qui nécessite l'utilisation d'un langage de modélisation adapté à ce paradigme. Pour cela, le meilleur choisi est l'UML (Unified Modeling Language), qui est reconnu pour sa capacité à représenter de manière visuelle et sémantique

la structure et le comportement de systèmes logiciels complexes.

UML est un langage de modélisation. La version actuelle, **UML 2.5**, propose **14 types de diagrammes** dont sept structurels et sept comportementaux.

UML n'étant pas une méthode, l'utilisation des diagrammes est laissée à l'appréciation de chacun. Le diagramme de classes est généralement considéré comme l'élément central d'UML. Des méthodes, telles que le processus unifié proposé par les créateurs originels de UML, utilisent plus systématiquement l'ensemble des diagrammes et axent l'analyse sur les cas d'utilisation (« use case ») pour développer par itérations successives un modèle d'analyse, un modèle de conception, et d'autres modèles. D'autres approches se contentent de modéliser seulement partiellement un système, par exemple certaines parties critiques qui sont difficiles à déduire du code. [9]



Figure 1.4: Logo UML `uml_logo`

1.6 Conclusion

Dans ce premier chapitre, nous avons exposé le contexte du projet, identifié les problèmes rencontrés, et proposé des critiques ainsi que des solutions potentielles. Nous avons également justifié le choix de la méthodologie de travail adoptée, en mettant en avant les éléments indicateurs qu'elle offre, ainsi que notre langage de modélisation. Dans le chapitre suivant, nous procéderons à l'analyse et à la spécification des besoins du projet.

ANALYSE ET SPÉCIFICATION DES BESOINS

Plan

1	Introduction	13
2	Spécification des besoins	13
3	Diagramme de cas d'utilisation global	15
4	Diagramme de Classes	16
5	Structure et découpage du projet	17
6	Architecture de l'Application	26
7	Choix Techniques et Environnement de Développement	29
8	Conclusion	38

2.1 Introduction

Dans ce chapitre, nous présentons une analyse des aspects fondamentaux de notre projet. En commençant par la précision des principaux acteurs et des fonctionnalités qui leur sont associées, passant par la définition des besoins fonctionnels et non fonctionnels, le diagramme de cas d'utilisation global et le backlog de produit. nous détaillerons l'architecture générale de l'application et l'environnement de travail.

2.2 Spécification des besoins

Dans cette section on va identifier les acteurs de l'application, les besoins fonctionnels et non fonctionnels.

2.2.1 Identification des acteurs

Un acteur est représenté par toute personne, équipe ou entité qui a un intérêt, une influence ou un impact sur le projet ou le produit développé.

- **Visiteur:** un utilisateur non inscrit ou non connecté qui peut accéder aux informations publiques de l'application ,sans pouvoir interagir avec les fonctionnalités réservées aux clients.
- **Client entreprise :** Représente toute organisation souhaitant acquérir ou évaluer des équipements informatiques reconditionnés. Il interagit avec la plateforme pour obtenir des recommandations personnalisées, simuler des scénarios de performance, consulter le catalogue, gérer son panier et soumettre des demandes de support.
- **Administrateur RedSys:** C'est le modérateur de l'application , chargée de gérer, superviser et maintenir la plateforme et la base de données, en assurant son bon fonctionnement et sa sécurité.
- **Employé RedSys:** Un collaborateur interne de RedSys disposant d'un accès limité à certaines fonctionnalités de gestion, notamment la mise à jour de son profil, le traitement des tickets assignées par l'administrateur.

2.2.2 Spécification des besoins fonctionnels

Nous précisons les besoins fonctionnels par acteur.

Visiteur

- Consulter librement le catalogue des équipements (serveurs, stockage, réseau) sans authentification ;

- Rechercher et filtrer les produits selon plusieurs critères (marque, prix, capacité, performance, type d'équipement) ;
- Consulter les détails d'un produit (caractéristiques techniques, état du produit, prix, images) ;
- Configurer un serveur en sélectionnant différents composants (RAM, stockage, processeur) selon ses besoins ;
- Ajouter des produits au panier et modifier les quantités ;
- Supprimer des produits du panier ;
- Consulter le récapitulatif du panier (prix total, liste des produits sélectionnés) ;

Client Entreprise

- S'authentifier de manière sécurisée à la plateforme ;
- Saisir ses besoins (nombre d'utilisateurs, type d'application, besoin en stockage) et obtenir une recommandation automatique de serveurs adaptés ;
- Consulter les détails des équipements recommandés (serveur, RAM, stockage conseillé, score de performance estimé) ;
- Simuler des scénarios de charge de travail et visualiser les indicateurs de performance (taux CPU, taux RAM, niveau de risque, stabilité) ;
- Parcourir le catalogue d'équipements, ajouter des produits au panier, modifier les quantités et valider une demande de commande ;
- Soumettre une demande de support (panne, réclamation) et suivre l'état de ses tickets.

Administrateur RedSys

- S'authentifier et accéder au tableau de bord d'administration ;
- Gérer les comptes utilisateurs : créer, modifier et désactiver les comptes employés ;
- Gérer le catalogue des équipements : ajout, modification, suppression et mise à jour des informations techniques ;
- Visualiser les indicateurs analytiques : types de serveurs les plus demandés, configurations les plus recommandées, taux de consultation, produits à forte et faible rotation ;
- Consulter et affecter les tickets de support clients aux employés ;
- Traiter les demandes de commandes clients et mettre à jour leur statut ;
- Gérer les rôles, les permissions d'accès et assurer la sécurité de la plateforme.

Employé

- S'authentifier à la plateforme avec les droits qui lui sont attribués ;
- Consulter et répondre aux tickets de support assignés selon ses permissions ;

2.2.3 Spécification des besoins non fonctionnels

Les besoins non fonctionnels désignent les critères qui définissent la qualité et les contraintes d'un système. Ils agissent indirectement sur les résultats du système et sur la productivité des utilisateurs.

- **Ergonomie** : La plateforme doit proposer une interface intuitive, claire et agréable, nécessitant un minimum de formation aussi bien pour les clients entreprises que pour les équipes internes de RedSys.
- **Responsivité** : L'application web doit offrir une expérience optimale sur une large gamme d'appareils et de résolutions d'écran, en adaptant dynamiquement sa mise en page.
- **Performance** : Le système doit afficher les recommandations et résultats de simulation en moins de 3 secondes ; l'interface doit rester fluide et réactive même lors de traitements intensifs côté serveur.
- **Disponibilité** : La plateforme doit maintenir un taux de disponibilité de 99,9% afin de garantir un accès continu et ininterrompu aux clients et aux équipes RedSys.
- **Scalabilité** : L'architecture doit supporter une croissance du catalogue d'équipements et une augmentation du nombre d'utilisateurs simultanés sans dégradation des performances.
- **Sécurité et Confidentialité** : Le système doit garantir la protection complète des données personnelles et commerciales. Tous les flux de données doivent être chiffrés via TLS, l'authentification assurée par JWT, et les accès strictement limités aux utilisateurs autorisés selon leurs rôles. Les mots de passe ne doivent jamais être stockés ou transmis en clair.
- **Traçabilité** : Toutes les actions critiques (modifications du catalogue, traitement des commandes, réponses aux tickets) doivent être journalisées et auditable à tout moment.
- **Maintenabilité et Évolutivité** : Le système doit être conçu avec une architecture modulaire et extensible, capable d'intégrer de futures évolutions telles que l'analyse prédictive du cycle de vie des équipements, la détection d'anomalies par IA et un modèle SaaS multi-clients.

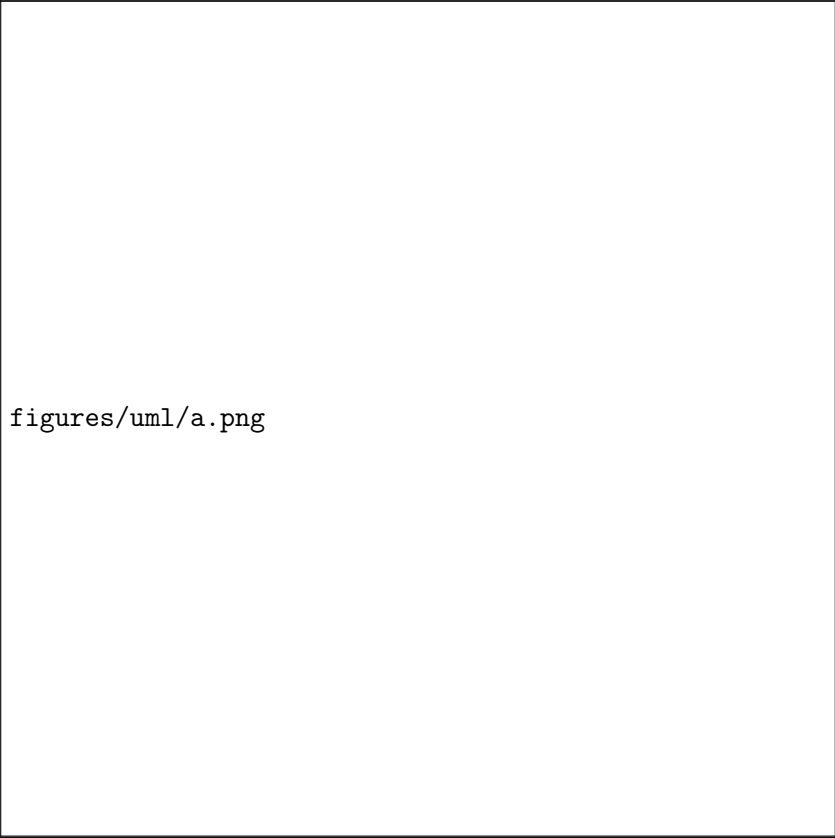
2.3 Diagramme de cas d'utilisation global

Les diagrammes des cas d'utilisation identifient les fonctionnalités fournies par le système (cas d'utilisation), les utilisateurs qui interagissent avec le système (acteurs), et l'interaction entre ces derniers. Les principaux objectifs des diagrammes des cas d'utilisation sont :

- Fournir une vue de haut-niveau de ce que fait le système.
- Identifier les utilisateurs (acteurs) du système.

La Figure 2.1 présente le diagramme de cas d'utilisation global:

Remarque: toutes les fonctionnalités nécessitent une authentification, sauf celles du visiteur.



figures/uml/a.png

Figure 2.1: Diagramme de cas d'utilisation global

2.4 Diagramme de Classes

Le diagramme de classes est un type de diagramme UML utilisé pour modéliser la structure statique d'un système. Il représente les classes du système, leurs attributs, leurs méthodes (opérations), ainsi que les relations entre ces classes.

La Figure 2.2 montre notre diagramme de classes.

figures/uml/.png

Figure 2.2: Diagramme de classes

2.5 Structure et découpage du projet

2.5.1 Identification de l'équipe SCRUM

L'équipe Scrum de ce projet est composée de :

- **Product Owner:** Yassine Khammassi
- **Scrum Master:** Mohamed Mehdi Kaouech
- **Developer:** Mohamed Aziz Merai

2.5.2 Backlog de produit

Le tableau 2.1 représente le backlog de produit de notre application .

Légende

Priorité MoSCoW :

Must - Must have	Fonctionnalité essentielle, requise pour le MVP
Should - Should have	Fonctionnalité importante mais non critique
Could - Could have	Fonctionnalité souhaitable mais non nécessaire
Won't - Won't have	Fonctionnalité non incluse dans cette version

Complexité :

1	Simple - Effort minimal, implémentation directe
2	Modérée - Effort modéré, complexité moyenne
3	Complexe - Effort significatif, implémentation complexe
5	Très complexe - Effort important, haute complexité

Tableau 2.1: Backlog de produit - Plateforme Intelligente de Recommandation de l'Infrastructure IT

ID	Feature	US ID	User Story	Priority	Complexi
1	Authentification	1.1	En tant qu'utilisateur, je veux me connecter de manière sécurisée à la plateforme.	Must	1
		1.2	En tant qu'utilisateur, je veux réinitialiser mon mot de passe via un processus sécurisé.	Should	2
		1.3	En tant qu'administrateur, je veux que le contrôle d'accès basé sur les rôles soit appliqué sur toute la plateforme.	Must	2

Suite à la page suivante...

ID	Feature	US ID	User Story	Priority	Complexi
2	Catalogue et Navigation	2.1	En tant que visiteur, je veux consulter librement le catalogue des équipements sans authentification.	Must	1
		2.2	En tant que visiteur, je veux rechercher et filtrer les produits selon plusieurs critères (marque, prix, type).	Must	2
		2.3	En tant que visiteur, je veux consulter les détails d'un produit (caractéristiques techniques, état, prix, images).	Must	1
		2.4	En tant que visiteur, je veux configurer un serveur en sélectionnant différents composants (RAM, stockage, processeur) selon mes besoins.	Should	3
		2.5	En tant que client, je veux parcourir le catalogue et ajouter des équipements au panier depuis les résultats de recommandation.	Must	1
3	Panier et Commandes	3.1	En tant que visiteur, je veux ajouter des produits au panier et modifier les quantités.	Must	1
		3.2	En tant que visiteur, je veux supprimer des produits de mon panier.	Must	1
		3.3	En tant que visiteur, je veux consulter le récapitulatif de mon panier (prix total, liste des produits sélectionnés).	Must	1
		3.4	En tant que client, je veux valider ma demande de commande et suivre son état après soumission.	Must	2

Suite à la page suivante...

ID	Feature	US ID	User Story	Priority	Complexi
4	Recommandation Intelligente	4.1	En tant que client, je veux saisir mes besoins (nombre d'utilisateurs, type d'application, stockage) pour obtenir une recommandation automatique de serveurs adaptés.	Must	3
		4.2	En tant que client, je veux consulter les détails des équipements recommandés (serveur, RAM, stockage conseillé, score de performance estimé).	Must	2
		4.3	En tant que client, je veux que le système utilise un service IA externe pour affiner les recommandations selon mon profil.	Must	3
		4.4	En tant qu'administrateur, je veux consulter les recommandations les plus fréquemment générées pour affiner le catalogue.	Could	2
5	Simulation de Performance	5.1	En tant que client, je veux simuler des scénarios de charge de travail sur un serveur pour évaluer ses capacités.	Must	3
		5.2	En tant que client, je veux visualiser les indicateurs de performance (taux CPU, taux RAM, niveau de risque, stabilité) après simulation.	Must	2
		5.3	En tant que client, je veux comparer les résultats de simulation entre plusieurs équipements.	Could	2
6	Gestion du Stock	6.1	En tant qu'administrateur, je veux gérer le catalogue CRUD des équipements (ajout, modification, suppression).	Must	2

Suite à la page suivante...

ID	Feature	US ID	User Story	Priority	Complexi
		6.2	En tant qu'administrateur, je veux mettre à jour les informations techniques des équipements.	Must	1
		6.3	En tant qu'administrateur, je veux consulter et traiter les commandes soumises par les clients.	Must	2
		6.4	En tant qu'administrateur, je veux recevoir une alerte lorsqu'un produit est en rupture de stock.	Should	2
7	Dashboard Analytique	7.1	En tant qu'administrateur, je veux visualiser les types de serveurs les plus demandés et les configurations les plus recommandées.	Must	3
		7.2	En tant qu'administrateur, je veux consulter le taux de consultation des produits et identifier les équipements à faible rotation.	Should	2
		7.3	En tant qu'administrateur, je veux filtrer, trier et exporter les données analytiques.	Should	2
		7.4	En tant qu'administrateur, je veux consulter l'historique des recommandations et l'évolution des tendances.	Should	3
8	Gestion des Tickets de Support	8.1	En tant que client, je veux soumettre une demande de support (panne, réclamation) via un formulaire dédié.	Must	2
		8.2	En tant que client, je veux suivre l'état de mes tickets (en cours, traité, fermé).	Must	1
		8.3	En tant qu'administrateur, je veux consulter les tickets clients et les affecter aux employés concernés.	Must	2

Suite à la page suivante...

ID	Feature	US ID	User Story	Priority	Complexi
		8.4	En tant qu'employé, je veux consulter et répondre aux tickets de support qui me sont assignés.	Must	2
8	Gestion des Utilisateurs	9.1	En tant qu'administrateur, je veux créer, modifier et désactiver les comptes employés.	Must	2
		9.2	En tant qu'administrateur, je veux gérer les rôles et les permissions d'accès de chaque utilisateur.	Must	2
		9.3	En tant qu'administrateur, je veux consulter les statistiques d'utilisation et les journaux d'audit de la plateforme.	Should	3
8	Évolutions Futures	10.1	En tant qu'administrateur, je veux que le système détecte automatiquement les anomalies de performance via une analyse IA avancée.	Could	5
		10.2	En tant qu'administrateur, je veux gérer plusieurs clients sous un modèle SaaS multi-tenant.	Could	5
		10.3	En tant que client, je veux recevoir des suggestions prédictives basées sur le cycle de vie des équipements.	Could	5

Tableau 2.1: Product Backlog

Résumé de l'Analyse

Total : 37 User Stories

Distribution des Priorités

- Must Have (M) : **24** user stories
- Should Have (S) : **7** user stories
- Could Have (C) : **6** user stories

- Won't Have (W) : **0** user story

Distribution des Complexités

- Complexité 1 : **10** user stories
- Complexité 2 : **18** user stories
- Complexité 3 : **6** user stories
- Complexité 5 : **3** user stories

2.5.3 Planification des Sprints

Une fois que le Backlog est préparé, l'étape suivante consiste à définir la durée du sprint et à répartir les éléments du backlog de produit sur un nombre de sprints.

Le tableau 2.2 montre la planification des sprints .

Release	Sprint	Durée (Jours)	Fonctionnalités
Release 1	Sprint 1	20	Authentification + Gestion des Utilisateurs (6 user stories – 10 pts) • Connexion sécurisée (client, employé, administrateur) • Réinitialisation du mot de passe • Contrôle d'accès basé sur les rôles • Création, modification et désactivation des comptes employés • Gestion des rôles et permissions d'accès • Consultation des journaux d'audit

Suite à la page suivante...

Release	Sprint	Durée (Jours)	Fonctionnalités
	Sprint 2	25	Catalogue, Navigation et Panier (8 user stories – 10 pts) • Consultation libre du catalogue sans authentification • Recherche et filtrage multicritères (marque, prix, capacité, type) • Fiche produit détaillée (caractéristiques, état, prix, images) • Configurateur de serveur (RAM, stockage, processeur) • Ajout, modification et suppression de produits du panier • Récapitulatif du panier (prix total, liste des produits) • Validation de commande et suivi de l'état

Suite à la page suivante...

Release	Sprint	Durée (Jours)	Fonctionnalités
	Sprint 3	20	Recommandation Intelligente + Simulation (7 user stories – 15 pts) • Saisie des besoins client (utilisateurs, type d'application, stockage) • Recommandation automatique via service IA externe • Affichage du serveur recommandé avec score de performance • Ajout au panier depuis les résultats de recommandation • Simulation de scénarios de charge de travail • Visualisation des indicateurs (CPU, RAM, risque, stabilité) • Comparaison des résultats entre plusieurs équipements

Suite à la page suivante...

Release	Sprint	Durée (Jours)	Fonctionnalités
Release 2	Sprint 4	25	Stock + Dashboard Analytique + Tickets (15 user stories – 27 pts) • Gestion CRUD des équipements et alertes rupture de stock • Consultation et traitement des commandes clients • Visualisation des serveurs les plus demandés • Taux de consultation et produits à faible rotation • Filtrage, tri et export des données analytiques • Historique des recommandations et tendances • Soumission et suivi des tickets de support par le client • Affectation des tickets aux employés par l'administrateur • Consultation et réponse aux tickets par l'employé
	Sprint 5	30	Évolutions Futures – IA Avancée + SaaS (3 user stories – 15 pts) • Détection automatique des anomalies de performance par IA • Gestion multi-clients sous architecture SaaS multi-tenant • Suggestions prédictives basées sur le cycle de vie des équipements
Résumé : 37 User Stories – 77 Points – 120 Jours			

Tableau 2.2: Planning des Sprints

2.6 Architecture de l'Application

Dans cette section, nous présentons l'architecture de l'application. Nous nous intéressons à l'architecture physique et logique de la solution.

2.6.1 Architecture Physique

L'architecture physique du système suit un modèle **N-Tiers**, qui sépare le système en couches distinctes, chacune dotée d'une responsabilité bien définie. Cette séparation garantit la scalabilité, la maintenabilité et des frontières claires entre les différents composants de la plateforme.

L'architecture physique est organisée en trois couches :

- **Couche Présentation :**

Cette couche englobe l'ensemble des interfaces côté client :

- **Application web Next.js** accessible par les visiteurs, les clients entreprises, les employés et les administrateurs RedSys depuis n'importe quel navigateur.

- **Couche Application :**

Cette couche héberge la logique métier centrale du système. Elle est constituée de deux composants principaux :

- **Serveur Next.js (API Routes)** : point d'entrée unique pour toutes les requêtes client, intégrant frontend et backend au sein du même projet. Il gère l'authentification JWT, le contrôle d'accès basé sur les rôles et l'orchestration de la logique métier ;
- **Service IA externe** : service indépendant dédié au moteur de recommandation intelligent, interrogé par le serveur Next.js via des appels API sécurisés pour générer les recommandations de serveurs et les scores de performance.

- **Couche Données :**

Cette couche est responsable de la persistance des données :

- **PostgreSQL** : base de données relationnelle principale stockant l'ensemble des données de la plateforme (utilisateurs, équipements, commandes, tickets, recommandations) ;
- **Prisma ORM** : interface de communication entre le serveur Next.js et PostgreSQL, offrant un accès typé, des migrations automatisées et une modélisation déclarative du schéma de données.

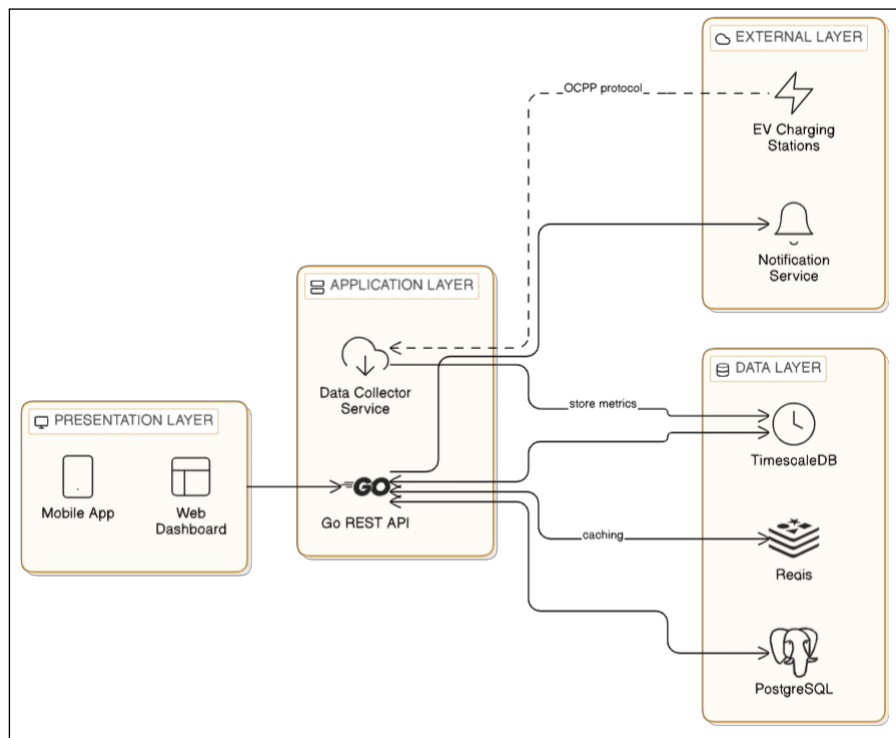


Figure 2.3: Physical Architecture (N-Tier)

2.6.2 Architecture Logique

L'architecture logique du système suit une architecture **Three-Tier** (trois niveaux), naturellement supportée par le framework Next.js, qui unifie frontend et backend au sein d'un même projet tout en maintenant une séparation claire des responsabilités :

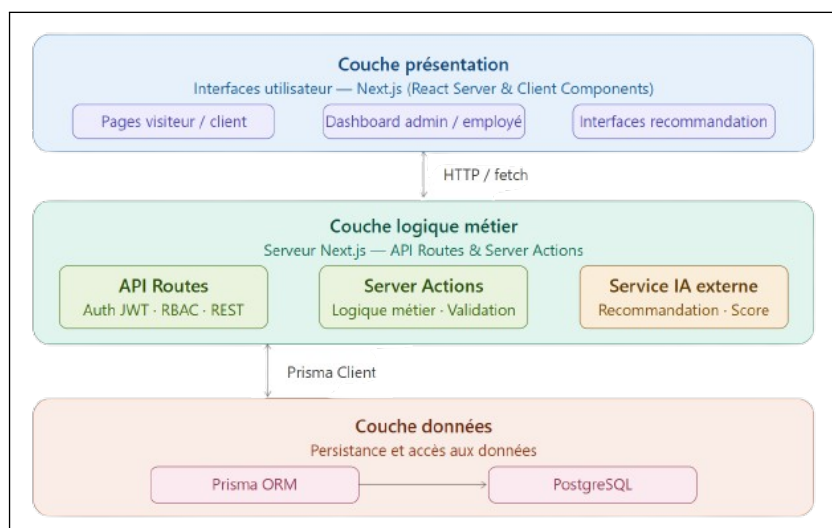


Figure 2.4: Architecture Logique (Three-Tier) — Next.js

Le pattern trois niveaux est appliqué comme suit:

- **Couche Présentation** : Pages et composants React rendus côté client, responsables de l'interface utilisateur et de la capture des interactions.
- **Couche Logique Métier** : API Routes et Server Actions Next.js exécutés côté serveur,

gérant l'authentification JWT, les règles métier et la communication avec le service IA externe.

- **Couche Données** : Prisma ORM interfacé avec PostgreSQL, assurant la persistance et l'accès structuré aux données.

2.7 Choix Techniques et Environnement de Développement

Cette section présente les technologies, frameworks, langages et outils sélectionnés pour le développement du système.

2.7.1 Langages de Programmation

2.7.1.1 HTML

HTML (HyperText Markup Language) est le langage de balisage standard pour la création de pages web [1]. Il décrit la structure du contenu web à l'aide d'un système d'éléments représentés par des balises. HTML5, la norme actuelle maintenue par le W3C, a introduit des éléments sémantiques, le support multimédia et des fonctionnalités d'accessibilité améliorées qui constituent l'épine dorsale structurelle des applications web modernes.



Figure 2.5: Logo HTML5

La figure 2.5 présente le logo de HTML5.

2.7.1.2 CSS

CSS (Cascading Style Sheets) est le langage utilisé pour décrire la présentation d'un document écrit en HTML [2]. Il contrôle la mise en page, les couleurs, la typographie, l'espacement et la responsivité sur différentes tailles d'écran. CSS3, la version actuelle, a introduit des fonctionnalités avancées telles que flexbox, grid layout, les animations et les media queries, permettant la création d'interfaces web entièrement responsives et visuellement cohérentes.



Figure 2.6: Logo CSS3

La figure 2.6 présente le logo de CSS3.

2.7.1.3 JavaScript

JavaScript est un langage de script léger et interprété, standardisé sous la spécification ECMAScript [3]. Il est le seul langage de programmation nativement supporté par les navigateurs web et constitue le fondement d'exécution des applications web interactives. Avec l'introduction de Node.js, JavaScript est également devenu largement utilisé côté serveur, et il constitue le socle de l'écosystème React et Next.js utilisés dans ce projet.



Figure 2.7: Logo JavaScript

La figure 2.7 présente le logo de JavaScript.

2.7.1.4 TypeScript

TypeScript est un langage de programmation open-source développé par Microsoft qui étend JavaScript en ajoutant un typage statique optionnel [4]. Il se compile en JavaScript pur et est entièrement compatible avec les bases de code JavaScript existantes. Le système de types de TypeScript permet la détection précoce des erreurs à la compilation, améliore la lisibilité du code et offre un puissant support IDE via l'autocomplétion et la documentation en ligne. Dans ce projet, TypeScript est utilisé à la fois côté frontend et côté backend au sein du projet Next.js.



Figure 2.8: Logo TypeScript

La figure 2.8 présente le logo de TypeScript.

2.7.1.5 Python

Python est un langage de programmation interprété, généraliste et open-source, reconnu pour sa lisibilité et sa richesse en bibliothèques scientifiques **python_lang**. Il est aujourd’hui le langage de référence pour le développement de solutions d’intelligence artificielle et de machine learning, grâce à des bibliothèques telles que scikit-learn, pandas et FastAPI. Dans ce projet, Python est utilisé pour le développement du service IA externe dédié au moteur de recommandation intelligente de serveurs.

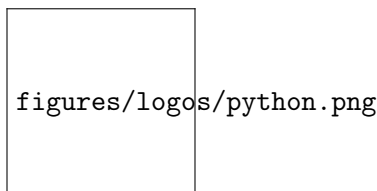


Figure 2.9: Logo Python

La figure 2.9 présente le logo de Python.

2.7.2 Couche API

2.7.2.1 API REST

Une API REST (Representational State Transfer) est une interface de programmation d’application qui respecte les contraintes architecturales REST **rest_arch**. Elle expose des ressources via des endpoints HTTP standard (GET, POST, PUT, DELETE) et repose sur des échanges de données au format JSON. Dans ce projet, l’API REST est implémentée au sein des API Routes de Next.js, servant de point de communication central entre le frontend et la base de données, tout en appliquant les règles d’authentification JWT et de contrôle d’accès basé sur les rôles.

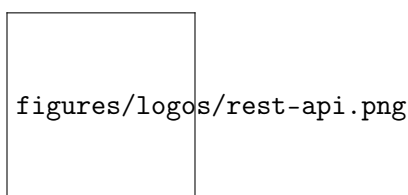


Figure 2.10: Illustration API REST

La figure 2.10 présente une illustration de l’API REST.

2.7.2.2 FastAPI

FastAPI est un framework web moderne et performant pour la création d’APIs avec Python **fastapi_framework**. Il est basé sur les annotations de types Python et génère automatiquement une documentation interactive via Swagger UI. FastAPI est utilisé dans ce projet pour exposer le service

IA externe sous forme d'une API REST indépendante, consommée par le serveur Next.js pour les fonctionnalités de recommandation et de simulation de performance.

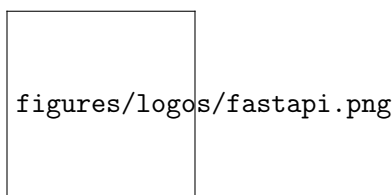


Figure 2.11: Logo FastAPI

La figure 2.11 présente le logo de FastAPI.

2.7.3 Frameworks et Bibliothèques

2.7.3.1 Next.js

Next.js est un framework React open-source développé par Vercel **nextjs_framework**. Il offre le rendu côté serveur (SSR), la génération de pages statiques (SSG), le routage basé sur les fichiers et les API Routes intégrées, permettant de développer à la fois le frontend et le backend au sein d'un seul et même projet. Next.js est aujourd'hui l'un des frameworks les plus adoptés pour la création d'applications web modernes, performantes et optimisées pour le référencement. Dans ce projet, Next.js constitue le socle unique de la plateforme RedSys, intégrant l'ensemble de la logique frontend et backend.

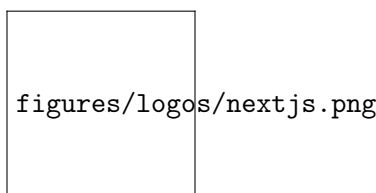


Figure 2.12: Logo Next.js

La figure 2.12 présente le logo de Next.js.

2.7.3.2 React

React est une bibliothèque JavaScript open-source développée et maintenue par Meta pour la construction d'interfaces utilisateur [5]. Elle a introduit une architecture basée sur des composants et un DOM virtuel qui met à jour efficacement uniquement les parties de l'interface ayant changé. Le flux de données unidirectionnel de React et son riche écosystème de bibliothèques complémentaires en font le framework frontend le plus largement adopté pour la création d'applications web complexes et interactives. Next.js s'appuie sur React comme fondation pour le rendu des interfaces côté serveur et côté client.

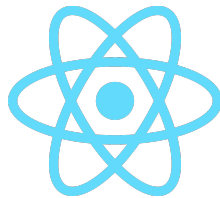


Figure 2.13: Logo React

La figure 2.13 présente le logo de React.

2.7.3.3 Tailwind CSS

Tailwind CSS est un framework CSS utilitaire open-source **tailwind_css**. Contrairement aux frameworks traditionnels comme Bootstrap, Tailwind ne fournit pas de composants prédéfinis, mais un ensemble de classes utilitaires de bas niveau permettant de construire des interfaces personnalisées directement dans le balisage HTML. Son approche *utility-first* permet un développement rapide et une personnalisation totale de l'interface, tout en maintenant un fichier CSS minimal en production grâce à son mécanisme de purge automatique.

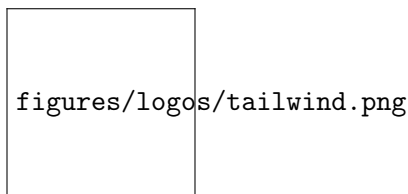


Figure 2.14: Logo Tailwind CSS

La figure 2.14 présente le logo de Tailwind CSS.

2.7.3.4 shadcn/ui

shadcn/ui est une collection de composants d'interface utilisateur réutilisables, accessibles et hautement personnalisables, construits sur Radix UI et stylisés avec Tailwind CSS **shadcn_ui**. Contrairement aux bibliothèques de composants traditionnelles, shadcn/ui ne s'installe pas comme une dépendance npm — les composants sont copiés directement dans le projet et peuvent être modifiés librement. Cette approche garantit un contrôle total sur le code, une accessibilité native et une cohérence visuelle parfaite avec le design system de la plateforme RedSys.

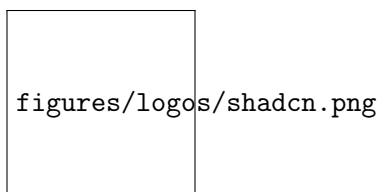


Figure 2.15: Logo shadcn/ui

La figure 2.15 présente le logo de shadcn/ui.

2.7.4 Base de Données et ORM

2.7.4.1 PostgreSQL

PostgreSQL est un système de gestion de base de données objet-relationnel puissant et open-source, avec plus de 35 ans de développement actif [6]. Il est entièrement conforme ACID et supporte des fonctionnalités avancées telles que les colonnes JSON, la recherche plein texte, les clés étrangères, les triggers et les procédures stockées. PostgreSQL est largement reconnu comme l’une des bases de données open-source les plus fiables et riches en fonctionnalités disponibles, ce qui en fait le choix naturel pour stocker l’ensemble des données de la plateforme RedSys.

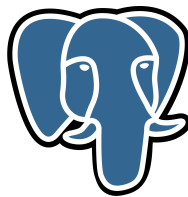


Figure 2.16: Logo PostgreSQL

La figure 2.16 présente le logo de PostgreSQL.

2.7.4.2 Prisma ORM

Prisma est un ORM (Object-Relational Mapper) open-source de nouvelle génération pour Node.js et TypeScript **prisma_orm**. Il génère un client de base de données entièrement typé à partir d’un schéma déclaratif, offrant une autocomplétion complète et une détection d’erreurs à la compilation. Prisma gère également les migrations de base de données de manière automatisée grâce à sa CLI intégrée, et s’interface nativement avec PostgreSQL. Son schéma déclaratif constitue la source de vérité unique pour la modélisation des entités du domaine dans ce projet.

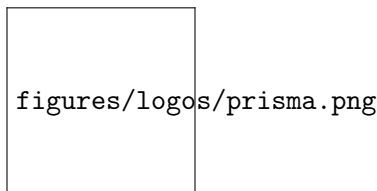


Figure 2.17: Logo Prisma ORM

La figure 2.17 présente le logo de Prisma ORM.

2.7.5 DevOps et Déploiement

2.7.5.1 Docker

Docker est une plateforme open-source pour le développement, la livraison et l'exécution d'applications dans des conteneurs [7]. Les conteneurs encapsulent une application et toutes ses dépendances dans une unité portable unique, garantissant un comportement cohérent entre les environnements de développement, de test et de production. Dans ce projet, Docker est utilisé pour conteneuriser l'application Next.js, le service IA Python et la base de données PostgreSQL.



Figure 2.18: Logo Docker

La figure 2.18 présente le logo de Docker.

2.7.5.2 Docker Compose

Docker Compose est un outil permettant de définir et d'orchestrer des applications multi-conteneurs à l'aide d'un seul fichier de configuration YAML [7]. Il permet de démarrer l'ensemble des services du projet (Next.js, service IA Python, PostgreSQL) avec une seule commande, en gérant automatiquement les dépendances entre conteneurs, les volumes persistants et les réseaux internes. Docker Compose simplifie considérablement la mise en place de l'environnement de développement local et de déploiement.

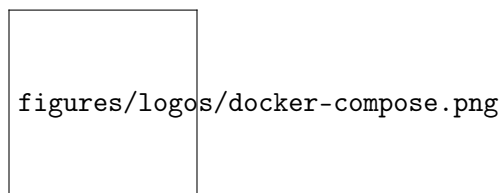
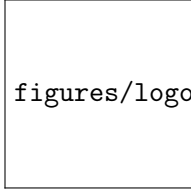


Figure 2.19: Logo Docker Compose

La figure 2.19 présente le logo de Docker Compose.

2.7.5.3 GitHub Actions

GitHub Actions est une plateforme d'intégration continue et de déploiement continu (CI/CD) intégrée nativement à GitHub [8]. Elle permet d'automatiser les workflows de build, de test et de déploiement directement depuis le dépôt de code. Dans ce projet, GitHub Actions est utilisé pour automatiser la vérification du code TypeScript, l'exécution des tests et le déploiement de l'application à chaque push sur la branche principale, garantissant ainsi une livraison continue et fiable.



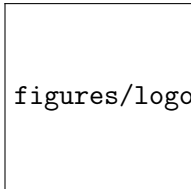
figures/logos/github-actions.png

Figure 2.20: Logo GitHub Actions

La figure 2.20 présente le logo de GitHub Actions.

2.7.5.4 Vercel

Vercel est la plateforme de déploiement cloud développée par les créateurs de Next.js **vercel_platform**. Elle offre un déploiement automatique à chaque push Git, un réseau de distribution de contenu (CDN) mondial, la gestion des variables d'environnement et un support natif de toutes les fonctionnalités Next.js (SSR, SSG, API Routes, Server Actions). Vercel constitue la solution de déploiement de référence pour les projets Next.js grâce à son intégration transparente et ses performances optimisées.



figures/logos/vercel.png

Figure 2.21: Logo Vercel

La figure 2.21 présente le logo de Vercel.

2.7.6 Outils de Développement

2.7.6.1 Visual Studio Code

Visual Studio Code est un éditeur de code gratuit et open-source développé par Microsoft [9]. Il supporte une large gamme de langages de programmation et offre un riche marketplace d'extensions couvrant TypeScript, React, Next.js, Prisma, Tailwind CSS et Python. Son architecture légère combinée à des fonctionnalités puissantes telles qu'IntelliSense, le terminal intégré, le débogueur et le support Git natif en font l'environnement de développement standard pour l'ensemble de ce projet.



Figure 2.22: Logo Visual Studio Code

La figure 2.22 présente le logo de Visual Studio Code.

2.7.6.2 Postman

Postman est une plateforme collaborative pour le développement et le test d'APIs [10]. Elle fournit une interface graphique intuitive permettant de concevoir, tester et documenter les endpoints REST. Dans ce projet, Postman est utilisé pour tester les API Routes de Next.js et les endpoints du service IA Python, en permettant d'envoyer des requêtes HTTP, d'inspecter les réponses et de valider le comportement des différents modules avant leur intégration.



Figure 2.23: Logo Postman

La figure 2.23 présente le logo de Postman.

2.7.6.3 pgAdmin

pgAdmin est l'outil d'administration et de gestion open-source de référence pour PostgreSQL **pgadmin_tool**. Il offre une interface graphique web permettant de visualiser la structure de la base de données, d'exécuter des requêtes SQL, de gérer les tables, les index et les relations, ainsi que de surveiller les performances du serveur PostgreSQL. Dans ce projet, pgAdmin est utilisé pour administrer la base de données de développement, vérifier les migrations Prisma et effectuer des requêtes d'analyse directement sur les données.

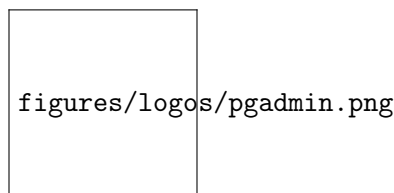


Figure 2.24: Logo pgAdmin

La figure 2.24 présente le logo de pgAdmin.

2.7.6.4 Git

Git est un système de contrôle de version distribué, gratuit et open-source, conçu pour gérer des projets de toute taille avec rapidité et efficacité [11]. Il permet à plusieurs développeurs de travailler simultanément sur la même base de code grâce au mécanisme de branches et de fusions, et maintient un historique complet de toutes les modifications apportées au projet.



Figure 2.25: Logo Git

La figure 2.25 présente le logo de Git.

2.7.6.5 GitHub

GitHub est une plateforme d'hébergement web pour les dépôts Git [8]. Elle fournit des outils pour la revue de code, le suivi des problèmes, la gestion de projet et l'intégration continue via GitHub Actions. GitHub sert de dépôt distant central pour ce projet et constitue le point d'entrée du pipeline CI/CD mis en place pour automatiser les tests et le déploiement de la plateforme RedSys.

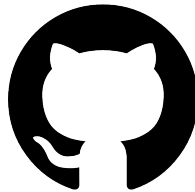


Figure 2.26: Logo GitHub

La figure 2.26 présente le logo de GitHub.

2.8 Conclusion

Le troisième chapitre décrit l'étude conceptuelle qui est divisée en deux parties, la première englobe la conception technique, alors que la seconde est destinée à la description des choix techniques et de l'environnement de travail. La description de la première release fera l'objet du chapitre suivant. Dans le deuxième chapitre, nous avons identifié les acteurs, les besoins fonctionnels et non fonctionnels de notre application, le Backlog du produit et la planification des sprints dans la 1ère partie alors que la seconde est destinée à la description des choix techniques et de l'environnement de travail. Dans les chapitres qui suivent, nous expliquerons de manière détaillée chaque release

RELEASE 1: MVP

Plan

1	Introduction	40
2	Release 1 Backlog	40
3	Refined Use Case Diagram for Release 1	42
4	Class Diagram of Release 1	43
5	Sprint 1: Authentication and User Management	43
6	Sprint 2: Mission Management and Mobile Application	50
7	Sprint 3: Data Collector Module	56
8	Conclusion	59

3.1 Introduction

In this chapter, we will present the different stages of the first release: the authentication module, user management, mission management and the mobile application for field technicians, as well as the automated data collector for EV charging stations.

3.2 Release 1 Backlog

ID	Feature	US ID	User Story	Priority	Complexi
1	Authentication + User Management	1.1	As a user, I want to log in securely to the system.	Must	1
		1.2	As a user, I want to reset my password via a secure process.	Should	2
		1.3	As an administrator, I want role-based access control enforced across all components.	Must	2
		1.4	As an administrator, I want to create technician and supervisor accounts.	Must	2
		1.5	As an administrator, I want to modify and deactivate user accounts.	Must	2
		1.6	As an administrator, I want to manage role assignments and access permissions.	Must	2
		1.7	As an administrator, I want to consult system usage statistics and audit logs.	Should	3
2	Mission Management + Mobile App	2.1	As a supervisor, I want to create and assign intervention missions to technicians.	Must	3
		2.2	As a supervisor, I want to track the lifecycle of each mission in real time (pending, in progress, completed, incident).	Must	3

Continued on next page...

ID	Feature	US ID	User Story	Priority	Complexi
		2.3	As a supervisor, I want to consult the full intervention history per station and per technician.	Should	2
		2.4	As a supervisor, I want to receive automatic alerts when a mission is overdue or an incident is reported.	Should	2
		2.5	As an administrator, I want to manage mission types and assignment rules.	Could	2
		2.6	As a technician, I want to view the list of missions assigned to me with their details.	Must	2
		2.7	As a technician, I want to update my mission status in real time with minimal interaction.	Must	2
		2.8	As a technician, I want to use the application in offline mode with automatic synchronization.	Must	3
		2.9	As a technician, I want to consult my personal intervention history.	Should	1
		2.10	As a technician, I want my geolocation to be optionally captured at the time of intervention.	Could	2
3	Data Collector Module	3.1	As a supervisor, I want the system to automatically connect to EV charging stations and retrieve their data.	Must	3
		3.2	As a supervisor, I want station data to be ingested continuously in real time.	Must	3
		3.3	As a supervisor, I want incoming data to be validated and anomalies handled automatically.	Must	2

Continued on next page...

ID	Feature	US ID	User Story	Priority	Complexi
		3.4	As a supervisor, I want collected data to be normalized and stored in a structured format.	Must	2
		3.5	As an administrator, I want to consult collection logs and monitor the collector's health.	Should	2

Tableau 3.1: Release 1 Backlog (MVP)

3.3 Refined Use Case Diagram for Release 1

Figure 3.1 shows the refined use case diagram for Release 1, covering the three main modules: authentication and user management, mission management and mobile application, and the data collector.



Figure 3.1: Refined Use Case Diagram – Release 1

3.4 Class Diagram of Release 1

Figure 3.2 shows the class diagram of Release 1, illustrating the main entities involved: User, Role, Mission, Station, Collector, and Log, along with their attributes, methods, and relationships.



Figure 3.2: Class Diagram – Release 1

3.5 Sprint 1: Authentication and User Management

3.5.1 Sprint 1 Objectives

The objective of Sprint 1 is to establish the security foundation of the platform. This sprint delivers the authentication system allowing technicians, supervisors, and administrators to log in securely, as well as the user management module enabling the administrator to create, modify, and deactivate accounts and manage role assignments.

3.5.2 Refined Use Case Diagram for Sprint 1

Figure 3.3 shows the refined use case diagram for Sprint 1, detailing the interactions between the Administrator and the Technician/Supervisor actors with the authentication and user management functionalities.



Figure 3.3: Refined Use Case Diagram – Sprint 1

3.5.3 Analysis and Design for Sprint 1

3.5.3.1 Use Case «Authenticate»

Textual description of the use case «Authenticate»

Use Case	Authenticate
Actor	Administrator, Supervisor, Technician
Objective	Allow system users to log in securely and access their respective workspaces according to their assigned role.
Precondition	The user must have a valid account with a username and password registered in the system.
Post-condition	Upon successful authentication, the user is redirected to their role-specific home interface.
Nominal Scenario	1. The user launches the application. 2. The system displays the authentication page. 3. The user enters their credentials (email and password). 4. The system validates the provided credentials. 5. The system generates a JWT token and establishes the session. 6. The user is redirected to their role-specific dashboard.
Alternative Scenario	A1 The entered credentials are incorrect. A2 The system displays an error message on the screen. A3 The system redirects the user back to the authentication page and returns to step 2 of the nominal scenario until authentication succeeds or the session is abandoned.

Tableau 3.2: Textual Description – Use Case «Authenticate»

Sequence Diagram – Use Case «Login»

Figure 3.4 shows the sequence diagram of the Login use case, illustrating the interactions between the user, the GraphQL API, the authentication service, and the database.



Figure 3.4: Sequence Diagram – Login

3.5.3.2 Use Case «User Management»

Textual description of the use case «User Management»

Use Case	User Management
Actor	Administrator
Objective	Allow the administrator to create, modify, deactivate, and manage role assignments for all user accounts on the platform.
Precondition	The administrator must be authenticated and have full administrative privileges.
Post-condition	The user account is successfully created, updated, or deactivated, and the changes are persisted in the database.
Nominal Scenario	1. The administrator accesses the user management interface. 2. The system displays the list of existing user accounts. 3. The administrator selects the desired action (create, modify, or deactivate). 4. The administrator fills in the required information. 5. The system validates the input data. 6. The system saves the changes and confirms the operation.
Alternative Scenario	A1 The provided data is invalid or incomplete. A2 The system displays a validation error message. A3 The system returns to step 4 of the nominal scenario until valid data is submitted.

Tableau 3.3: Textual Description – Use Case «User Management»**Sequence Diagram – Use Case «Add User»**

Figure 3.5 shows the sequence diagram of the Add User use case, illustrating the interactions between the administrator, the GraphQL API, and the database during the account creation process.



Figure 3.5: Sequence Diagram – Add User

3.5.4 Realization of Sprint 1

Figure 3.6 and Figure 3.7 show the implemented interfaces for Sprint 1, covering the authentication screen and the user management dashboard.



Figure 3.6: Sprint 1 – Login Interface



Figure 3.7: Sprint 1 – User Management Interface

3.6 Sprint 2: Mission Management and Mobile Application

3.6.1 Sprint 2 Objectives

The objective of Sprint 2 is to deliver the mission management module and the technician mobile application. Supervisors will be able to create, assign, and track intervention missions in real time, while technicians will be able to consult their assigned missions, update their status, and operate in offline mode with automatic synchronization.

3.6.2 Refined Use Case Diagram for Sprint 2

Figure 3.8 shows the refined use case diagram for Sprint 2, detailing the interactions between the Supervisor and Technician actors with the mission management and mobile application functionalities.



Figure 3.8: Refined Use Case Diagram – Sprint 2

3.6.3 Analysis and Design for Sprint 2

3.6.3.1 Use Case «Mission Management»

Textual description of the use case «Mission Management»

Use Case	Mission Management
Actor	Supervisor, Administrator
Objective	Allow supervisors to create, assign, and track intervention missions for field technicians across all EV charging stations.
Precondition	The supervisor must be authenticated. At least one technician account and one charging station must exist in the system.
Post-condition	The mission is successfully created, assigned to a technician, and its status is tracked in real time.
Nominal Scenario	1. The supervisor accesses the mission management interface. 2. The system displays the list of existing missions. 3. The supervisor creates a new mission by specifying the station, description, priority, and deadline. 4. The supervisor assigns the mission to a technician. 5. The system saves the mission and notifies the assigned technician. 6. The supervisor monitors the mission lifecycle in real time.
Alternative Scenario	A1 No technician is available for assignment. A2 The system displays a warning and allows the mission to be saved as unassigned. A3 The supervisor can assign the mission later from the mission detail view.

Tableau 3.4: Textual Description – Use Case «Mission Management»**Sequence Diagram – Use Case «Add Mission»**

Figure 3.9 shows the sequence diagram of the Add Mission use case, illustrating the interactions

between the supervisor, the GraphQL API, the notification service, and the database.



Figure 3.9: Sequence Diagram – Add Mission

3.6.3.2 Use Case «Mobile Application»

Textual description of the use case «Mobile Application»

Use Case	Update Mission Status
Actor	Technician
Objective	Allow the technician to update the status of an assigned mission directly from the mobile application in real time.
Precondition	The technician must be authenticated and have at least one mission assigned to them.
Post-condition	The mission status is updated in the system and the supervisor is notified of the change in real time.
Nominal Scenario	1. The technician opens the mobile application. 2. The system displays the list of assigned missions. 3. The technician selects a mission and views its details. 4. The technician updates the mission status (e.g., In Progress, Completed, Incident). 5. The system saves the update and synchronizes with the server. 6. The supervisor receives a real-time notification of the status change.
Alternative Scenario	A1 The technician has no internet connection. A2 The update is stored locally in offline mode. A3 Upon reconnection, the system automatically synchronizes the pending update with the server.

Tableau 3.5: Textual Description – Use Case «Update Mission Status»

Sequence Diagram – Use Case «Update Mission Status»

Figure 3.10 shows the sequence diagram of the Update Mission Status use case, illustrating the interactions between the technician, the mobile application, the GraphQL API, and the database, including the offline synchronization scenario.



Figure 3.10: Sequence Diagram – Update Mission Status

3.6.4 Realization of Sprint 2

Figure 3.11 and Figure 3.12 show the implemented interfaces for Sprint 2, covering the mission management dashboard and the technician mobile application.



Figure 3.11: Sprint 2 – Mission Management Interface



Figure 3.12: Sprint 2 – Technician Mobile Application

3.7 Sprint 3: Data Collector Module

3.7.1 Sprint 3 Objectives

The objective of Sprint 3 is to implement the automated data collector service. This component connects securely to EV charging stations, ingests their data continuously in real time, validates and normalizes the collected measurements, and stores them in the TimescaleDB time-series database for later consultation and analysis.

3.7.2 Refined Use Case Diagram for Sprint 3

Figure 3.13 shows the refined use case diagram for Sprint 3, detailing the interactions between the Collector, the Administrator, and the EV Charging Station with the data ingestion and monitoring functionalities.



Figure 3.13: Refined Use Case Diagram – Sprint 3

3.7.3 Analysis and Design for Sprint 3

3.7.3.1 Use Case «Data Collection»

Textual description of the use case «Data Collection»

Use Case	Data Collection
Actor	Data Collector (System), Administrator
Objective	Automatically connect to EV charging stations, retrieve their real-time data, validate and normalize it, and store it in a structured format for supervision and analysis.
Precondition	The collector service is running and the EV charging stations are reachable via their native protocol.
Post-condition	The collected data is validated, normalized, and persisted in TimescaleDB. Any detected anomaly triggers an alert.
Nominal Scenario	1. The collector service starts and reads the list of configured stations. 2. The collector establishes a secure connection to each station. 3. The station returns its current data (status, measurements, signals). 4. The collector validates the received data against defined rules. 5. The collector normalizes the data into a unified format. 6. The data is stored in TimescaleDB and published to Redis for real-time propagation.
Alternative Scenario	A1 The collector cannot reach a station. A2 The system logs the connection failure and marks the station as offline. A3 The collector retries the connection according to a configured retry policy.

Tableau 3.6: Textual Description – Use Case «Data Collection»

Sequence Diagram – Use Case «Data Collection»

Figure 3.14 shows the sequence diagram of the Data Collection use case, illustrating the interactions between the collector service, the EV charging station, the validation layer, TimescaleDB, and Redis.



Figure 3.14: Sequence Diagram – Data Collection

3.7.4 Realization of Sprint 3

Figure 3.15 shows the implemented collector service logs and monitoring interface delivered at the end of Sprint 3.



Figure 3.15: Sprint 3 – Data Collector Monitoring Interface

3.8 Conclusion

This chapter presented the complete realization of Release 1. The three sprints delivered the core foundation of the platform: a secure authentication and user management system, a mission management module coupled with a cross-platform mobile application for field technicians, and an automated data collector service for real-time EV station monitoring. The following chapter will present the realization of Release 2, which introduces the web supervision dashboard, the alert and notification system, and the platform’s future evolutions.

RELEASE 2: IMPROVEMENTS

Plan

1	Introduction	61
2	Release 2 Backlog	61
3	Refined Use Case Diagram for Release 2	62
4	Class Diagram of Release 2	63
5	Sprint 4: Web Supervision Dashboard and Alerts & Notifications	63
6	Sprint 5: AI and Multi-tenancy	72
7	Conclusion	78

4.1 Introduction

In this chapter, we will present the different stages of the second release: the web supervision dashboard, the alert and notification system, and the future evolutions of the platform including the multi-tenant SaaS architecture and AI-based anomaly detection.

4.2 Release 2 Backlog

ID	Feature	US ID	User Story	Priority	Complexity
4	Web Supervision + Alerts & Notifications	4.1	As a supervisor, I want to access a real-time dashboard showing the status of all EV charging stations.	Must	3
		4.2	As a supervisor, I want to view detailed information per station (operational, faulty, offline, under maintenance).	Must	2
		4.3	As a supervisor, I want to search, filter, sort, and export station and mission data.	Should	2
		4.4	As a supervisor, I want to consult historical data and trend visualizations.	Should	3
		4.5	As a supervisor, I want to configure alerts for critical events (station failure, data anomaly, missed mission).	Should	2
		4.6	As a supervisor, I want to receive real-time alerts when a station reports a critical event.	Must	3
		4.7	As a technician, I want to receive push notifications when a new mission is assigned to me.	Should	2
		4.8	As a supervisor, I want to configure notification thresholds and alert recipients.	Could	2

Continued on next page...

ID	Feature	US ID	User Story	Priority	Complexi
5	AI & Multi-tenancy	5.1	As a supervisor, I want the system to automatically detect anomalies using AI-based analysis.	Could	5
		5.2	As an administrator, I want to manage multiple station fleets under a SaaS multi-tenant model.	Could	5
		5.3	As a supervisor, I want predictive maintenance suggestions based on collected station data.	Could	5

Tableau 4.1: Release 2 Backlog (Improvements)

4.3 Refined Use Case Diagram for Release 2

Figure 4.1 shows the refined use case diagram for Release 2, covering the web supervision dashboard, the alert and notification system, and the AI and multi-tenancy evolutions.



Figure 4.1: Refined Use Case Diagram – Release 2

4.4 Class Diagram of Release 2

Figure 4.2 shows the class diagram of Release 2, extending the Release 1 model with the entities introduced in this release: Alert, Notification, Report, and the multi-tenant Tenant entity.



Figure 4.2: Class Diagram – Release 2

4.5 Sprint 4: Web Supervision Dashboard and Alerts & Notifications

4.5.1 Sprint 4 Objectives

The objective of Sprint 4 is to deliver the web supervision dashboard and the alert and notification system. Supervisors will be able to monitor the entire EV station fleet in real time, consult detailed station information, search and export data, and receive configurable alerts for critical events. Technicians will also receive push notifications upon new mission assignments.

4.5.2 Refined Use Case Diagram for Sprint 4

Figure 4.3 shows the refined use case diagram for Sprint 4, detailing the interactions between the Supervisor and Technician actors with the web dashboard and alert functionalities.



Figure 4.3: Refined Use Case Diagram – Sprint 4

4.5.3 Analysis and Design for Sprint 4

4.5.3.1 Use Case «Web Supervision Dashboard»

Textual description of the use case «Web Supervision Dashboard»

Use Case	Web Supervision Dashboard
Actor	Supervisor
Objective	Allow supervisors to monitor the entire EV charging station fleet and all ongoing missions in real time through a centralized web interface.
Precondition	The supervisor must be authenticated. The data collector must be active and ingesting station data.
Post-condition	The supervisor has a real-time view of the fleet status and can take informed decisions based on the displayed data.
Nominal Scenario	1. The supervisor logs in and accesses the supervision dashboard. 2. The system fetches the latest station and mission data via GraphQL. 3. The dashboard displays the global fleet status with per-station indicators (operational, faulty, offline, maintenance). 4. The supervisor applies filters or searches for a specific station or mission. 5. The supervisor consults detailed information and historical trends for a selected station. 6. The supervisor exports the filtered data if needed.
Alternative Scenario	A1 The data collector is offline and no recent data is available. A2 The system displays a warning banner indicating the last successful data sync timestamp. A3 The supervisor is redirected to the collector health monitoring view.

Tableau 4.2: Textual Description – Use Case «Web Supervision Dashboard»

Sequence Diagram – Use Case «View Station Status»

Figure 4.4 shows the sequence diagram of the View Station Status use case, illustrating the interactions between the supervisor, the React dashboard, the GraphQL API, Redis, and TimescaleDB.



Figure 4.4: Sequence Diagram – View Station Status

4.5.3.2 Use Case «Alert Management»

Textual description of the use case «Alert Management»

Use Case	Alert Management
Actor	Supervisor
Objective	Allow supervisors to receive, consult, and configure real-time alerts triggered by critical events such as station failures, data anomalies, or overdue missions.
Precondition	The supervisor must be authenticated and alert rules must be configured in the system.
Post-condition	The supervisor is notified of critical events in real time and can acknowledge or escalate alerts as needed.
Nominal Scenario	1. A critical event is detected by the system (station failure, anomaly, overdue mission). 2. The alert service generates an alert record and publishes it via Redis pub/sub. 3. The GraphQL subscription propagates the alert to connected dashboard clients. 4. The supervisor receives the alert in real time on the dashboard. 5. The supervisor consults the alert details and takes the appropriate action. 6. The supervisor acknowledges or resolves the alert.
Alternative Scenario	A1 The supervisor is not connected to the dashboard at the time of the alert. A2 The alert is stored in the database and displayed as unread upon the supervisor's next login. A3 A push notification is also sent if configured.

Tableau 4.3: Textual Description – Use Case «Alert Management»**Sequence Diagram – Use Case «Receive Alert»**

Figure 4.5 shows the sequence diagram of the Receive Alert use case, illustrating the event

propagation chain from the collector service through Redis and the GraphQL subscription to the supervisor's dashboard.



Figure 4.5: Sequence Diagram – Receive Alert

4.5.3.3 Use Case «Push Notification»

Textual description of the use case «Push Notification»

Use Case	Push Notification
Actor	Technician
Objective	Notify the technician via a push notification when a new mission is assigned to them, ensuring immediate awareness regardless of their current activity.
Precondition	The technician must have the mobile application installed and push notifications enabled on their device.
Post-condition	The technician receives a push notification and can directly access the mission details from the notification.
Nominal Scenario	1. A supervisor assigns a new mission to the technician. 2. The system saves the mission and triggers the notification service. 3. The notification service sends a push notification to the technician's device via FCM. 4. The technician receives the notification on their mobile device. 5. The technician taps the notification and is redirected to the mission detail screen in the mobile application.
Alternative Scenario	A1 The technician's device is unreachable or offline. A2 The notification is queued by FCM and delivered upon reconnection. A3 The mission appears as unread in the technician's mission list on next application launch.

Tableau 4.4: Textual Description – Use Case «Push Notification»**Sequence Diagram – Use Case «Push Notification»**

Figure 4.6 shows the sequence diagram of the Push Notification use case, illustrating the

interactions between the supervisor, the GraphQL API, the notification service, FCM, and the technician's mobile device.



Figure 4.6: Sequence Diagram – Push Notification

4.5.4 Realization of Sprint 4

Figure 4.7 and Figure 4.8 show the implemented interfaces for Sprint 4, covering the web supervision dashboard and the alert management interface.



Figure 4.7: Sprint 4 – Web Supervision Dashboard



Figure 4.8: Sprint 4 – Alert Management Interface

4.6 Sprint 5: AI and Multi-tenancy

4.6.1 Sprint 5 Objectives

The objective of Sprint 5 is to deliver the future evolution features of the platform: AI-based anomaly detection for proactive station monitoring, predictive maintenance suggestions based on historical time-series data, and the foundation of a multi-tenant SaaS architecture enabling EntityService's client to manage multiple independent station fleets under a single platform instance.

4.6.2 Refined Use Case Diagram for Sprint 5

Figure 4.9 shows the refined use case diagram for Sprint 5, detailing the interactions between the Supervisor, the Administrator, and the AI engine with the anomaly detection, predictive maintenance, and multi-tenancy functionalities.



Figure 4.9: Refined Use Case Diagram – Sprint 5

4.6.3 Analysis and Design for Sprint 5

4.6.3.1 Use Case «AI-Based Anomaly Detection»

Textual description of the use case «AI-Based Anomaly Detection»

Use Case	AI-Based Anomaly Detection
Actor	System (AI Engine), Supervisor
Objective	Automatically detect abnormal patterns in the collected station data using AI-based analysis and notify the supervisor of potential issues before they escalate into failures.
Precondition	Sufficient historical station data must be available in TimescaleDB to train and run the anomaly detection model.
Post-condition	Detected anomalies are recorded as alerts and communicated to the supervisor in real time via the dashboard and notification system.
Nominal Scenario	1. The AI engine periodically queries TimescaleDB for recent station metrics. 2. The model analyzes the data and identifies deviations from expected patterns. 3. A detected anomaly is flagged and an alert record is created. 4. The alert is propagated to the supervisor via GraphQL subscription. 5. The supervisor reviews the anomaly details and decides on the appropriate action (e.g., scheduling a maintenance mission).
Alternative Scenario	A1 The model confidence score is below the defined threshold. A2 The anomaly is flagged as a low-confidence warning rather than a critical alert. A3 The supervisor can review and confirm or dismiss the warning manually.

Tableau 4.5: Textual Description – Use Case «AI-Based Anomaly Detection»

Sequence Diagram – Use Case «Anomaly Detection»

Figure 4.10 shows the sequence diagram of the Anomaly Detection use case, illustrating the interactions between the AI engine, TimescaleDB, the alert service, Redis, and the supervisor’s dashboard.



Figure 4.10: Sequence Diagram – AI-Based Anomaly Detection

4.6.3.2 Use Case «Multi-Tenant Management»

Textual description of the use case «Multi-Tenant Management»

Use Case	Multi-Tenant Management
Actor	Administrator
Objective	Allow the platform administrator to create and manage multiple independent tenant instances, each with its own isolated data, users, and station fleet, under a shared SaaS infrastructure.
Precondition	The administrator must be authenticated with super-admin privileges at the platform level.
Post-condition	A new tenant is provisioned with its own PostgreSQL schema, isolated data, and a dedicated administrator account.
Nominal Scenario	1. The super-admin accesses the tenant management portal. 2. The super-admin creates a new tenant by providing its name and configuration. 3. The system automatically provisions a new PostgreSQL schema for the tenant. 4. The system creates a dedicated administrator account for the new tenant. 5. The new tenant administrator can log in and begin configuring their station fleet and users independently.
Alternative Scenario	A1 The tenant name already exists in the system. A2 The system displays a conflict error and prompts the super-admin to choose a unique identifier. A3 The provisioning process is retried with the corrected information.

Tableau 4.6: Textual Description – Use Case «Multi-Tenant Management»

Sequence Diagram – Use Case «Provision New Tenant»

Figure 4.11 shows the sequence diagram of the Provision New Tenant use case, illustrating the interactions between the super-admin, the GraphQL API, the tenant provisioning service, and the PostgreSQL database.



Figure 4.11: Sequence Diagram – Provision New Tenant

4.6.4 Realization of Sprint 5

Figure 4.12 and Figure 4.13 show the implemented interfaces for Sprint 5, covering the anomaly detection dashboard and the tenant management portal.



Figure 4.12: Sprint 5 – AI Anomaly Detection Dashboard



Figure 4.13: Sprint 5 – Multi-Tenant Management Portal

4.7 Conclusion

This chapter presented the complete realization of Release 2. Sprint 4 delivered the web supervision dashboard and the alert and notification system, providing supervisors with a real-time, comprehensive view of the entire EV station fleet. Sprint 5 introduced the platform's future evolutions, including AI-based anomaly detection, predictive maintenance, and the foundation of a multi-tenant SaaS architecture. Together, the two releases constitute a complete, scalable, and production-ready integrated system for EV charging station management and technician mission supervision.

General Conclusion

This report presents our end-of-studies project carried out within the framework of our internship at **EntityService**, as part of our academic journey at **Tek-Up University** in computer engineering. This project offered us the opportunity to apply the theoretical and technical knowledge acquired throughout our studies, particularly in the development of modern, integrated, and real-time software systems.

Our project, entitled «*Conception et développement d’une plateforme de gestion et de supervision en temps réel des bornes de recharge pour véhicules électriques*», responds to a growing operational need: replacing a fragmented, spreadsheet-based workflow with a reliable, scalable, and professional platform capable of managing EV charging station data and technician missions in real time.

Through the development of this platform, we designed and implemented an integrated system composed of three core components: an automated data collector that continuously ingests real-time measurements from EV charging stations, a cross-platform mobile application dedicated to field technicians for managing their intervention missions, and a web supervision dashboard providing back-office teams with a comprehensive, real-time view of the entire station fleet.

Throughout this project, we adopted an agile development approach based on the **Scrum** methodology, enabling incremental and iterative delivery of the system across clearly defined sprints. This approach helped us structure our work into two releases and five sprints, each delivering functional and testable increments of the platform.

During our internship at EntityService, we successfully realized the following modules:

- Secure authentication and role-based user management for administrators, supervisors, and technicians;
- Mission management with real-time lifecycle tracking and assignment to field technicians;
- A cross-platform mobile application with offline mode and automatic synchronization;
- An automated data collector service for continuous EV station data ingestion, validation, and storage;
- A web supervision dashboard with real-time station monitoring, configurable alerts, and data visualization.

Thanks to these achievements, the platform enables EntityService’s client to significantly improve the supervision of their station fleet, reduce human error, gain operational traceability, and enhance the productivity of both field and back-office teams. Furthermore, the integration of a full observability stack combining **ELK**, **Prometheus**, and **Grafana** ensures efficient monitoring and prepares the

platform for future scaling.

However, this project represents a first version that opens the way to several possible evolutions. Among the planned improvements are the completion of the multi-tenant SaaS architecture enabling the management of multiple independent station fleets under a single platform instance, the integration of AI-based anomaly detection for proactive maintenance, and the addition of predictive maintenance suggestions based on historical time-series data collected by the platform.

In summary, this project constitutes a solid and innovative foundation for addressing the current challenges of EV charging station management. It delivers a reliable, scalable, and future-ready architecture, and offers a promising framework for further developments aimed at accompanying the digital transformation of the electric mobility sector.

Webography

- [1] W3C, « HTML5 Specification », World Wide Web Consortium, Tech. Rep., 2014. [Online]. Available: <https://www.w3.org/TR/html5>
- [2] W3C, « CSS Snapshot », World Wide Web Consortium, Tech. Rep., 2023. [Online]. Available: <https://www.w3.org/TR/css-2023>
- [3] Ecma International, « ECMAScript 2023 Language Specification », Ecma International, Tech. Rep., 2023. [Online]. Available: <https://www.ecma-international.org/publications-and-standards/standards/ecma-262>
- [4] Microsoft, *TypeScript: JavaScript With Syntax For Types*, 2012. [Online]. Available: <https://www.typescriptlang.org>
- [5] Meta Platforms, *React: A JavaScript Library for Building User Interfaces*, 2013. [Online]. Available: <https://react.dev>
- [6] PostgreSQL Global Development Group, *PostgreSQL: The World's Most Advanced Open Source Database*, 1996. [Online]. Available: <https://www.postgresql.org>
- [7] Docker Inc., *Docker: Accelerated Container Application Development*, 2013. [Online]. Available: <https://www.docker.com>
- [8] GitHub Inc., *GitHub: Where the World Builds Software*, 2008. [Online]. Available: <https://github.com>
- [9] Microsoft, *Visual Studio Code*, 2015. [Online]. Available: <https://code.visualstudio.com>
- [10] Postman Inc., *Postman API Platform*, 2012. [Online]. Available: <https://www.postman.com>
- [11] Linus Torvalds, *Git: Distributed Version Control System*, 2005. [Online]. Available: <https://git-scm.com>

Résumé

Ce rapport présente le travail effectué dans le cadre du projet de fin d'études à Tek-Up University, réalisé au sein d'EntityService.

L'objectif est la conception et le développement d'une plateforme intégrée de gestion et supervision en temps réel des bornes de recharge pour véhicules électriques. La plateforme comprend trois composants : un collecteur automatique de données, une application mobile pour les techniciens de terrain, et un tableau de bord web de supervision.

Le système repose sur une architecture moderne combinant un backend en **Go** avec API **GraphQL**, un dashboard **React**, une application **React Native**, et une stack d'observabilité (**ELK**, **Prometheus**, **Grafana**). Le projet a été mené selon la méthodologie **Scrum**, organisée en deux releases et cinq sprints.

Mots clés : Véhicule Électrique, Borne de Recharge, Supervision Temps Réel, Go, GraphQL, React, Scrum

Abstract

This report summarizes the end-of-studies project for the computer engineering diploma at Tek-Up University, completed at EntityService.

The objective is the design and development of an integrated platform for real-time management and supervision of electric vehicle charging stations. The platform consists of three components: an automatic data collector, a mobile app for field technicians, and a web supervision dashboard. The system is built on a modern architecture combining a **Go** backend with **GraphQL** API, a **React** dashboard, a **React Native** mobile app, and an observability stack (**ELK**, **Prometheus**, **Grafana**). The project followed **Scrum** agile methodology, organized into two releases and five sprints.

Keywords : Electric Vehicle, Charging Station, Real-Time Supervision, Go, GraphQL, React, React Native, Scrum
